HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

PROJECT 01

HỆ ĐA AGENT CHO BỆNH TIỂU ĐƯỜNG

PHÁT TRIỂN CÁC HỆ THỐNG THÔNG MINH

Giảng viên:	PGS. TS. Trần Đình Quế
Sinh viên:	Kiều Trường Giang
Mã sinh viên:	B23DCCN254
Nhóm học phần:	06
Nhóm thực hiện:	02

Hà Nội, tháng 11 năm 2026

Mục lục

1. Giới thiệu 1

1.1. Bài toán và sản phẩm 1

1.2. Phạm vi đầu ra 1

1.3. Mục tiêu đánh giá 1

2. Agent và cơ chế phối hợp 2

2.1. Vai trò xử lý 2

2.2. Reinforcement Learning kết hợp luật 2

2.3. Giới hạn của kiến trúc 3

3. Kiến trúc tổng quan 4

3.1. Nhánh dữ liệu 4

3.2. Nhánh tri thức 4

3.3. Luồng người dùng 5

4. Nguồn dữ liệu và quy trình chuẩn bị 6

4.1. Phân biệt dữ liệu mô hình với tài liệu tri thức 6

4.2. Nguồn và phiên bản bộ phân loại 6

4.3. Từ điển đầu vào và giá trị thiếu 7

4.4. Phép chia và thông tin tái lập 8

4.5. Nguồn bộ hồi quy và ý nghĩa mục tiêu 8

4.6. Nguồn KB và nguồn đang thực sự được dùng 9

4.7. Từ nguồn tới kết quả hiển thị 12

5. Các thành phần và thực nghiệm 13

5.1. Phân loại 13

5.2. Hồi quy 13

5.3. Kết quả kiểm tra 13

5.4. Truy xuất và đồ thị 14

5.5. Phân tích sai số theo lớp 14

5.6. Phân tích phần dư hồi quy 15

5.7. Hợp đồng dữ liệu và API 16
5.8. Kết quả kiểm thử tích hợp 16
5.9. Đối chiếu với sơ đồ yêu cầu 17

6. KB, KG và RAG trong bộ sản phẩm nộp 18
6.1. Các thành phần có thể kiểm tra độc lập 18
6.2. Agent đồ thị tham gia luồng trả lời 19
6.2.1. Trách nhiệm và hợp đồng thông điệp 19
6.2.2. Đối chiếu căn cứ trước khi đưa vào mô hình 19
6.2.3. Mất kết nối và thứ tự ưu tiên 20
6.3. Xuất và chạy lại bộ nộp 20
6.3.1. Kiểm tra nguồn trước khi xuất 20
6.3.2. Chỉ mục và cấu hình RAG 20
6.3.3. Lệnh sử dụng 20
6.4. Kết quả kiểm tra 21
6.5. Phạm vi và giới hạn 22

7. Code và demo 23
7.1. Địa chỉ và cách chạy 23
7.2. Kịch bản trình diễn 23
7.3. Hạn chế và hướng hoàn thiện 24
7.4. Bản triển khai công khai và bản local 24

8. Hội thoại cục bộ và thực nghiệm mô hình ngôn ngữ 25
8.1. Mô hình dùng trong bản triển khai 25
8.2. Đường đi của một lượt hỏi 25
8.2.1. Phân loại an toàn và điều phối 25
8.2.2. Truy xuất và tạo phản hồi 25
8.2.3. Đối chiếu và sửa phản hồi 26
8.3. Nhận diện ý định và học cách trình bày 27
8.3.1. Bộ nhận diện câu hỏi tiếng Việt 27
8.3.2. Phản hồi theo từng chủ đề 27
8.4. Thực nghiệm tinh chỉnh Qwen3 1.7B 28
8.4.1. Dữ liệu và cấu hình 28
8.4.2. Kết quả đo trên máy thực nghiệm 28

8.4.3. Kết quả và quyết định sử dụng 29
8.5. Vận hành và kịch bản trình diễn 29

9. Phân loại an toàn và kiểm thử 30
9.1. Mục tiêu và vị trí trong hệ thống 30
9.2. Mức độ ưu tiên và dữ kiện sử dụng 30
9.3. Phủ định và ngữ cảnh 31
9.4. Bảo vệ phản hồi 31
9.5. Kiểm thử hoàn toàn nội bộ 31
9.6. Giới hạn áp dụng 32

10. Giao diện và kịch bản sử dụng 33
10.1. Thiết kế giao diện và cách đọc kết quả 33
10.1.1. Trợ lý kiến thức 33
10.1.2. Thực nghiệm dữ liệu: phân loại 34
10.1.3. Thực nghiệm dữ liệu: hồi quy 35
10.1.4. Đồ thị tri thức và đối chiếu Neo4j 36
10.1.5. Dữ liệu và nguồn 37
10.2. Thiết kế và vận hành chatbot tiểu đường 38
10.2.1. Ngữ cảnh, phạm vi và cơ chế trả lời 38
10.2.2. Kiểm tra chỉ số trước khi giải thích 39
10.2.3. Nối mô hình với hội thoại và so sánh đầu vào 39
10.2.4. Tra cứu KB và sử dụng Neo4j 40
10.2.5. Kiểm thử bổ sung và thiết bị 40
10.2.6. Hội thoại tiếp nối và đối chiếu nhiều chỉ số 41

A. Mã nguồn tái lập 43
A.1. scripts/ingest_graph.py 43
A.2. scripts/train_models.py 43
A.3. scripts/train_policy.py 44
A.4. app/agents.py 45
A.5. app/conversation.py 45
A.6. app/main.py 50
A.7. app/safety.py 55
A.8. app/retrieval.py 57

A.9. app/graph_knowledge.py 59
A.10. scripts/query_knowledge_export.py 60
A.11. scripts/test_scenarios.py 61
A.12. scripts/test_chatbot.py 62

Danh mục hình

3.1. Hai nhánh hội tụ về điều phối và trợ giúp, dựng lại từ sơ đồ trên bảng. 4

5.1. Ma trận nhầm lẫn của MLP trên tập kiểm tra phân tầng. 15
5.2. Dự đoán và phần dư của MLP trên 89 mẫu kiểm tra. 15

8.1. Luồng hội thoại local với bước phân loại an toàn trước truy xuất. 27
8.2. Loss huấn luyện theo bước và validation theo epoch. Điểm đầu validation thuộc mô hình nền. 28

9.1. Nhánh an toàn trên web local: hành động ưu tiên được đưa trước phần diễn giải. 32

10.1. Hội thoại trên bản local và cách trình bày nguồn tham khảo. 33
10.2. Màn hình phân loại: mẫu đầu vào và nhãn do checkpoint MLP trả về. 34
10.3. Màn hình hồi quy: biểu mẫu riêng và điểm tiến triển nghiên cứu. 35
10.4. Toàn bộ đồ thị trên web với nút xem nhanh và điều khiển phóng to. 36
10.5. Neo4j Browser trả về cùng 143 khái niệm và 86 quan hệ. 36
10.6. Màn hình công bố nguồn gốc và phạm vi hai bộ dữ liệu. 37
10.7. Giao diện trợ lý tiểu đường với thanh điều hướng, vùng hội thoại và tài liệu đối chiếu. 38
10.8. Bố cục điện thoại: điều hướng hai hàng và hội thoại một cột. 41

Danh mục bảng

4.1. Danh mục dữ liệu thực sự dùng trong GlucoGuide. 6
4.2. Kiểm kê tám biến đầu vào trên toàn bộ CSV, trước tiền xử lý. 7
4.3. Nguồn thực sự được 46 tài liệu trong KB tham chiếu. 9

5.1. Phân loại trên 10.000 mẫu kiểm tra. 14
5.2. Hồi quy trên 89 mẫu kiểm tra. 14

6.1. Các tệp chính trong bộ nộp tri thức. 18
6.2. Kết quả truy xuất và kiểm tra bộ nộp. 21

8.1. Kiểm tra hình thức đầu ra trên hai nhóm test, mỗi nhóm 8 câu. 29

9.2. Các nhóm kiểm thử an toàn chính. 31

Chương 1

Giới thiệu

1.1. Bài toán và sản phẩm

Project xây dựng GlucoGuide, một ứng dụng web tích hợp hai nhánh xử lý của sơ đồ học phần. Nhánh dữ liệu sử dụng mô hình mạng nơ-ron nhiều lớp để phân loại nhãn nguy cơ tổng hợp và dự đoán điểm tiến triển bệnh trong dữ liệu nghiên cứu. Nhánh tri thức kế thừa kho tài liệu của tiểu luận 02, tổ chức các khái niệm cùng quan hệ trên Neo4j và truy xuất văn bản theo câu hỏi. Kết quả hai nhánh được trình bày cùng nguồn bằng chứng và nhật ký xử lý.

GlucoGuide phát triển từ các thực nghiệm học phần, bổ sung cơ chế phối hợp giữa mô hình, truy xuất và phản hồi. Toàn bộ kết quả đánh giá được lưu cùng mã nguồn để thuận tiện đối chiếu.

1.2. Phạm vi đầu ra

Hai bộ dữ liệu có định nghĩa nhãn khác nhau. Bộ phân loại gồm các nhãn High, Moderate và Low của dữ liệu tổng hợp. Bộ hồi quy gồm những người đã mắc bệnh và một thước đo tiến triển sau một năm. Vì vậy không thể ghép một nhãn High với một điểm hồi quy để suy ra xác suất mắc bệnh. Ứng dụng dùng hai biểu mẫu riêng và công bố phạm vi ngay tại đầu ra.

Bản công khai sử dụng tổng hợp trích đoạn xác định. Câu trả lời được ghép từ các đoạn văn truy xuất được và thông tin về phạm vi mô hình; không gọi mô hình ngôn ngữ bên ngoài. Bản cục bộ dùng Qwen3 4B lượng tử hóa để diễn đạt trên tài liệu truy xuất; các yêu cầu có nguy cơ được phân loại trước và xử lý bằng luật. Hai chế độ đều phụ thuộc chất lượng và độ phủ của kho tài liệu.

1.3. Mục tiêu đánh giá

Phân loại được đo bằng accuracy và macro-F1. Hồi quy được đo bằng MAE và hệ số xác định. Truy xuất cần kiểm tra cả tài liệu được chọn và quan hệ có nguồn trên Neo4j. Kiểm tra ứng dụng tập trung vào các tình huống có bằng chứng, thiếu bằng chứng, thiếu đầu vào, giá trị không hợp lệ và cơ sở dữ liệu đồ thị không hoạt động.

Chương 2

Agent và cơ chế phối hợp

2.1. Vai trò xử lý

Hệ thống phân chia trách nhiệm thành an toàn, mô hình, truy xuất văn bản, đồ thị tri thức, chính sách trình bày, ngôn ngữ và kiểm chứng phản hồi. Các agent có handler được gọi qua MessageBus; nhật ký lưu người gửi, người nhận, loại thông điệp và nội dung. Các thành phần hoạt động trong một tiến trình FastAPI dưới điều phối tập trung, chưa phải các dịch vụ tự chủ độc lập.

Một thông điệp REQUEST thể hiện yêu cầu xử lý. INFORM ghi nhận kết quả và FAILURE thể hiện thành phần không thực hiện được. Khi Neo4j không hoạt động, hệ thống ghi FAILURE và chỉ dùng các đoạn văn cục bộ, không giả lập kết quả đồ thị. Khi thiếu đầu vào mô hình, hệ thống trả lại danh sách trường cần bổ sung trước khi suy luận.

2.2. Reinforcement Learning kết hợp luật

Chính sách được học trong môi trường mô phỏng luồng xử lý, không mô phỏng hiệu quả điều trị. Trạng thái gồm hai biến nhị phân: đã có kết quả mô hình và đã có bằng chứng truy xuất. Bốn trạng thái tương ứng với bốn tổ hợp của hai biến này. Ba hành động là hỏi làm rõ, trả lời có nguồn và giải thích mô hình kèm nguồn.

Hàm thưởng bằng một nếu hành động phù hợp trạng thái, bằng âm một trong trường hợp còn lại. Khi chưa có bằng chứng, hành động mục tiêu là hỏi làm rõ. Khi có bằng chứng nhưng chưa có kết quả mô hình, hành động mục tiêu là trả lời có nguồn. Khi có cả hai, hành động mục tiêu là giải thích mô hình và dẫn nguồn. Đây là hàm thưởng do người thiết kế quy định; không phải đánh giá của bệnh nhân hay chuyên gia.

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r - Q(s, a)].$$

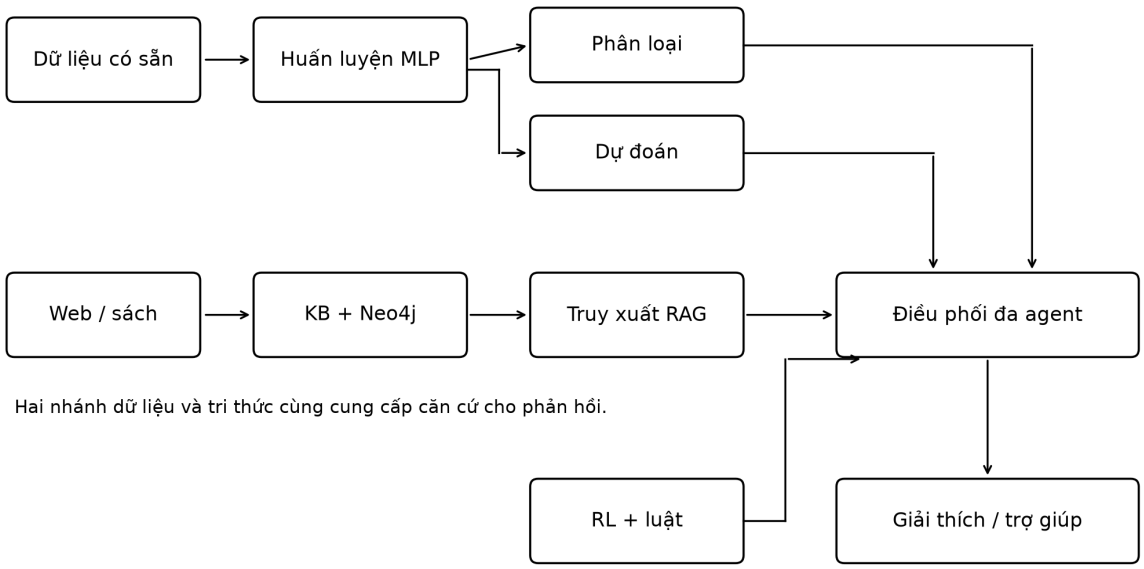
Mỗi episode kết thúc sau một hành động, nên không có thành phần bootstrap tương lai. Cấu hình gồm 3.000 episode, seed 254, tốc độ học 0,15 và xác suất khám phá 0,2. Vì bài toán chỉ có một bước, thí nghiệm minh họa cơ chế học giá trị hành động nhưng chưa chứng minh khả năng lập kế hoạch dài hạn. Luật thiếu bằng chứng luôn ép hành động hỏi làm rõ, kể cả khi chính sách học đề xuất hành động khác.

2.3. Giới hạn của kiến trúc

Mỗi agent có handler, bộ nhớ trong lượt và thông điệp yêu cầu/phản hồi có thể kiểm tra. Hệ thống chưa có bộ nhớ hội thoại bền vững, cơ chế tranh biện hoặc triển khai agent thành dịch vụ độc lập. Nhật ký nhiều vai trò giúp quan sát xử lý, nhưng không tự chứng minh mức tự chủ hoặc lợi ích so với một pipeline đơn giản hơn.

Chương 3

Kiến trúc tổng quan



Hình 3.1: Hai nhánh hội tụ về điều phối và trợ giúp, dựng lại từ sơ đồ trên bảng.

3.1. Nhánh dữ liệu

DB có sẵn cung cấp dữ liệu cho quá trình huấn luyện. Pipeline phân loại gồm điền trung vị, chuẩn hóa và MLP. Pipeline hồi quy chuẩn hóa dữ liệu rồi suy luận bằng MLP. Hai đầu ra đều đi tới vùng phản hồi của ứng dụng. Trong lúc phục vụ, mô hình được nạp từ checkpoint; ứng dụng không huấn luyện lại theo câu hỏi của người dùng.

3.2. Nhánh tri thức

Nguồn tri thức là các đoạn văn đã được tổ chức trong tiểu luận 02. Mỗi tài liệu có mã, tiêu đề, từ khóa, văn bản, định danh nguồn và các bộ ba. Project bảo toàn những trường này khi chuyển sang kho tri thức mới. Việc cập nhật hướng dẫn y tế từ web chưa tự động hóa; phiên bản kho tài liệu phải được xem như một bản chụp dữ liệu cần rà soát trước khi mở rộng phạm vi sử dụng.

Kho tri thức gồm 46 đoạn từ 19 nguồn được tham chiếu. Mọi đoạn đều có quan hệ gắn với nguồn và trích đoạn làm căn cứ; tổng cộng 86 bộ ba được nạp vào Neo4j local. Nội dung gồm xét nghiệm, ăn uống, vận động, theo dõi, thuốc, các loại tiểu đường và trợ giúp sinh hoạt.

Một bộ ba gồm khái niệm đầu, tên quan hệ và khái niệm cuối. Neo4j lưu hai đầu dưới nhãn Concept; quan hệ RELATED_TO có thuộc tính label biểu diễn tên quan hệ, document biểu diễn tài liệu và evidence biểu diễn đoạn văn làm căn cứ. Ràng buộc duy nhất theo tên khái niệm và thao tác MERGE giúp nạp lại dữ liệu mà không tạo các bản sao giống hệt.

3.3. Luồng người dùng

Yêu cầu được phân loại an toàn trước khi chuyển đến truy xuất. Yêu cầu mô hình phải đi qua kiểm tra các trường trước khi suy luận. Văn bản và kết quả đồ thị được chuyển sang đối chiếu; chính sách học và luật xác định kiểu phản hồi. Giao diện hiển thị câu trả lời, tài liệu nguồn và các bước xử lý. Lớp hội thoại nhận tối đa 64 thông điệp gần nhất để giải quyết câu hỏi tiếp nối; chỉ dùng câu hỏi người dùng làm gợi ý chủ đề, không dùng lời trả lời cũ làm căn cứ y tế. Các trường của hai mô hình được tách riêng để tránh biến đổi sai đơn vị hoặc tự suy diễn giá trị còn thiếu.

Sau khi chạy một trong hai mô hình, nút “Giải thích kết quả” gửi lại loại bài toán và các giá trị đã nhập đến agent mô hình. Chatbot trả lời bằng nhãn hoặc điểm thực tế của mẫu, kèm chỉ số đánh giá ngoài mẫu và giới hạn sử dụng. Nhánh này dùng giải thích xác định, không để Qwen suy đoán ý nghĩa lâm sàng của nhãn tổng hợp hoặc điểm tiến triển.

Chương 4

Nguồn dữ liệu và quy trình chuẩn bị

4.1. Phân biệt dữ liệu mô hình với tài liệu tri thức

Project sử dụng hai bộ dữ liệu học có giám sát và một kho tài liệu. Bộ phân loại cung cấp nhãn High, Moderate, Low; bộ hồi quy cung cấp một điểm tiến triển sau một năm; KB cung cấp văn bản giải thích và nguồn. Ba thành phần này có mục tiêu khác nhau. Dữ liệu mô hình không tự trở thành hướng dẫn y khoa, còn tài liệu y khoa không được coi là nhãn huấn luyện cho hai MLP.

Bảng 4.1: Danh mục dữ liệu thực sự dùng trong GlucoGuide.

Thành phần	Quy mô	Nơi lấy và nơi sử dụng
Phân loại	50.000 dòng, 41 cột gốc; dùng 8 biến	Kaggle, Diabetes Risk Prediction Dataset (50K patients), tài khoản mobeenfatimah. ZIP lưu tại `data/raw/diabetes.zip`; dùng trong `scripts/train_models.py`.
Hồi quy	442 dòng, 10 biến và target số	Bộ diabetes đi kèm scikit-learn, đọc bằng `load_diabetes(scaled=False)`. Không dùng ZIP của bài toán phân loại.
Tri thức	46 đoạn; 19 nguồn được tham chiếu	Hai tệp KB được đọc chung; 86 bộ ba liên kết toàn bộ chủ đề trên Neo4j local.

4.2. Nguồn và phiên bản bộ phân loại

Trang công bố bộ dữ liệu:

https://www.kaggle.com/datasets/mobeenfatimah/diabetes-risk-prediction-dataset-50k-patients.

Tệp nằm trong ZIP có tên `diabetes_risk_prediction_dataset.csv`. Báo cáo sử dụng đúng bản chụp dữ liệu đã lưu trên máy, không tự thay bằng bộ 100.000 bản ghi của bài tham khảo. Tệp kiểm kê `results/data_audit.json` lưu SHA-256 của ZIP để phân biệt các phiên bản tải về.

Đơn vị quan sát là một bản ghi tổng hợp, không phải hồ sơ lâm sàng được project thu thập. Cột mục tiêu là `Diabetes_Risk`. Kiểm kê trực tiếp thấy 36.593 nhãn High, 12.937 Moderate và 470 Low, tương ứng 73,186%, 25,874% và 0,94%. Một mô hình chỉ đoán High sẽ có accuracy 73,186% trên toàn bộ bản chụp dữ liệu; vì vậy phải đọc accuracy cùng macro-F1 và recall từng lớp.

Không có nhãn thiếu và không có dòng trùng hoàn toàn trên 41 cột. Kết luận về dòng trùng không chứng minh các hồ sơ là những cá nhân độc lập, nhất là khi bảng có cột định danh. Pipeline hiện giữ nguyên 50.000 dòng, không có bước loại trùng hay loại ngoại lệ. Những đặc điểm này được công bố để người đọc biết chính xác dữ liệu nào đã đi vào phép chia.

4.3. Từ điển đầu vào và giá trị thiếu

Bảng 4.2: Kiểm kê tám biến đầu vào trên toàn bộ CSV, trước tiền xử lý.

Biến	Số ô thiếu	Min / trung vị / max
Age	497	18 / 54 / 90
BMI	0	11.9 / 30 / 61.7
HbA1c	1974	4.5 / 8.5 / 12.5
Fasting_Blood_Sugar	0	65 / 142.6 / 220
Blood_Glucose	959	70 / 159.9 / 250
Blood_Pressure_Systolic	0	90 / 140 / 190
HDL	1000	25 / 57.3 / 90
LDL	1000	50 / 135.1 / 220

Các cột được chọn lần lượt là tuổi, BMI, HbA1c, đường huyết lúc đói, glucose, huyết áp tâm thu, HDL và LDL. Min, trung vị, max ở Bảng 4.2 là thống kê mô tả của bản chụp dữ liệu, không phải khoảng bình thường hay ngưỡng chẩn đoán. Trên giao diện, nút điền mẫu lấy đúng giá trị của bộ dữ liệu. Người dùng không nên tự thay một chỉ số ở đơn vị khác rồi coi kết quả tương đương.

Những trường `Patient_ID`, `Diabetes_Risk_Score`, cột khuyến nghị sức khỏe và `Doctor_Consultation_Needed` không được chọn làm đặc trưng. Đặc biệt, điểm nguy cơ và khuyến nghị có thể gắn trực tiếp với quy trình tạo nhãn; đưa chúng vào dễ khiến bài toán trở thành khôi phục thông tin đã gần như có sẵn. Việc bỏ các trường này giảm một nguy cơ rò rỉ, nhưng không chứng minh mọi phụ thuộc trong dữ liệu tổng hợp đã được loại bỏ.

Trung vị dùng để điền thiếu được học trên tập train của pipeline, không dùng trung vị của toàn bộ bảng ở Bảng 4.2. Sau đó StandardScaler học trung bình và độ lệch chuẩn trên train. Pipeline chứa cả imputer, scaler và mô hình được lưu bằng joblib, nhờ đó web dùng cùng phép

biến đổi với quá trình huấn luyện. Khi nhập từ web, các trường hiện vẫn bắt buộc; imputer chủ yếu phục vụ giá trị thiếu trong dữ liệu huấn luyện và kiểm tra.

4.4. Phép chia và thông tin tái lập

Phân loại chia 80/20, phân tầng theo nhãn, seed 254: 40.000 dòng train và 10.000 dòng test. Chỉ số dòng bắt đầu từ 0 và nhãn train/test được lưu tại `results/classification_split.csv`. Tập test có 7.319 High, 2.587 Moderate và 94 Low. Kiểm kê dữ liệu không huấn luyện lại mô hình và không thay kết quả đã báo cáo.

MLP bật early stopping với phần validation nội bộ của thư viện. Validation này nằm bên trong train ngoài, nhưng imputer và scaler đã fit trước khi MLP tách validation. Đây là điểm cần cải thiện nếu thiết kế một thí nghiệm chọn mô hình nghiêm ngặt. Test ngoài không được dùng để fit pipeline. Báo cáo không nhận rằng đã có ba tập train/validation/test tách trước toàn bộ tiền xử lý.

4.5. Nguồn bộ hồi quy và ý nghĩa mục tiêu

Nguồn phân phối:

`https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html`.

Nguồn gốc được thư viện dẫn tới nghiên cứu Least Angle Regression và trang dữ liệu thống kê:

`https://www4.stat.ncsu.edu/~boos/var.select/diabetes.html`.

Bộ dữ liệu gồm 442 người đã mắc bệnh với mục tiêu là một đại lượng tiến triển sau một năm; mô tả của thư viện không cho phép đổi mục tiêu đó thành đường huyết hoặc xác suất mắc bệnh[13].

Project gọi `scaled=False`, sau đó chuẩn hóa bằng scaler của riêng train. Mười cột gồm age, sex, bmi, bp và s1 đến s6. age là tuổi, bmi là chỉ số khối cơ thể, bp là huyết áp trung bình. Theo metadata, s1 là cholesterol toàn phần, s2 LDL, s3 HDL, s4 tỷ lệ cholesterol/HDL, s5 được mô tả thận trọng là có thể là log triglyceride, s6 là chỉ số đường huyết. sex được giữ nguyên mã nghiên cứu; không dùng mã này để tự chọn ngưỡng y khoa.

Bộ hồi quy không có giá trị thiếu trong ma trận đặc trưng. Phép chia ngẫu nhiên seed 254 giữ 353 mẫu train và 89 mẫu test, ghi tại `results/regression_split.csv`. Hai mô hình cùng dùng phép chia này. Chỉ số dòng trong `results/regression_predictions.csv` cho phép truy lại mục tiêu thật và dự đoán của từng mẫu test.

4.6. Nguồn KB và nguồn đang thực sự được dùng

Bảng 4.3: Nguồn thực sự được 46 tài liệu trong KB tham chiếu.

Nguồn	Mã tài liệu	Số đoạn
NIDDK · Xét nghiệm & chẩn đoán	a1c, testing	2
CDC · Xử trí hạ đường huyết	low, urgent	2
CDC · Lập kế hoạch bữa ăn	plate, carb, fruit, meal_example, rice_portion, fruit_whole	6
WHO · Diabetes	activity, symptoms, overview	3
NIDDK · Quản lý bệnh tiểu đường	medicine, monitor, care_questions, glucose_log, new_diagnosis	5
scikit-learn · Dữ liệu diabetes	meanpressure, triglyceride, demographics, model	4
AHA · Hiểu số đo huyết áp	bloodpressure	1
MedlinePlus · Các chỉ số cholesterol	lipids, hdl, ldl, ratio	4
Bộ dữ liệu tổng hợp Diabetes Risk Prediction 50K	classification_scope	1
NIDDK · Diabetes & Foot Problems	foot_care	1
CDC · Managing Sick Days	sick_day	1
NIDDK · Quick Reference on UACR & GFR	kidney_checks	1
CDC · Get Active	activity_plan	1
NIDDK · What Is Diabetes?	types_compare, prediabetes, pregnancy_diabetes, complications	4
NIDDK · Healthy Living with Diabetes	drinks, meal_timing, alcohol	3
NIDDK · Insulin, Medicines & Treatments	metformin_info, insulin_info, missed_medicine	3
CDC · Diabetes and Mental Health	diabetes_distress, stress_glucose	2
ADA · Facts About Diabetes	not_contagious	1
ADA · Diabetes Remission Criteria	remission_info	1

NIDDK · Xét nghiệm & chẩn đoán

https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis

CDC · Xử trí hạ đường huyết

https://www.cdc.gov/diabetes/treatment/treatment-low-blood-sugar-hypoglycemia.html

CDC · Lập kế hoạch bữa ăn

https://www.cdc.gov/diabetes/healthy-eating/diabetes-meal-planning.html

WHO · Diabetes

https://www.who.int/news-room/fact-sheets/detail/diabetes

NIDDK · Quản lý bệnh tiểu đường

https://www.niddk.nih.gov/health-information/diabetes/overview/managing-diabetes

scikit-learn · Dữ liệu diabetes

https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html

AHA · Hiểu số đo huyết áp

https://www.heart.org/en/health-topics/high-blood-pressure/understanding-blood-pressure-readings

MedlinePlus · Các chỉ số cholesterol

https://medlineplus.gov/cholesterollevelswhatyouneedtoknow.html

Bộ dữ liệu tổng hợp Diabetes Risk Prediction 50K

https://www.kaggle.com/datasets/mobeenfatimah/diabetes-risk-prediction-dataset-50k-patients

NIDDK · Diabetes & Foot Problems

https://www.niddk.nih.gov/health-information/diabetes/overview/preventing-problems/foot-problems

CDC · Managing Sick Days

https://www.cdc.gov/diabetes/living-with/managing-sick-days.html

NIDDK · Quick Reference on UACR & GFR

https://www.niddk.nih.gov/health-information/professionals/advanced-search/quick-reference-uacr-gfr

CDC · Get Active

https://www.cdc.gov/diabetes/living-with/physical-activity.html

NIDDK · What Is Diabetes?

https://www.niddk.nih.gov/health-information/diabetes/overview/what-is-diabetes

NIDDK · Healthy Living with Diabetes

https://www.niddk.nih.gov/health-information/diabetes/overview/healthy-living-with-diabetes

NIDDK · Insulin, Medicines & Treatments

https://www.niddk.nih.gov/health-information/diabetes/overview/insulin-medicines-treatments

CDC · Diabetes and Mental Health

https://www.cdc.gov/diabetes/living-with/mental-health.html

ADA · Facts About Diabetes

https://diabetes.org/about-diabetes/diabetes-myths

ADA · Diabetes Remission Criteria

https://diabetes.org/newsroom/international-experts-outline-diabetes-remission-diagnosis-criteria

Kho tri thức được đọc từ hai tệp JSON, gồm 46 đoạn và 19 nguồn thực sự được tham chiếu. Mỗi đoạn có mã, tiêu đề, từ khóa, văn bản, nguồn và bộ ba quan hệ. Các liên kết gốc được lưu để người đọc đối chiếu.

Luồng Web/sách đến KB trong sơ đồ được hiện thực ở mức biên soạn bản chụp dữ liệu có nguồn. Project chưa có nút upload PDF, trình thu thập web, cơ chế cập nhật định kỳ hay bộ chia chương sách tự động. Các PDF tham khảo của sinh viên chỉ phục vụ bố cục và phân tích thiết kế; chúng không được nạp làm nguồn tư vấn y khoa. Khi thay KB cần rà nội dung, cập nhật JSON, chạy lại script nạp Neo4j và khởi động lại backend để xây lại ma trận TF-IDF.

Mỗi chủ đề có ít nhất một bộ ba được đối chiếu với văn bản nguồn. Sau khi nạp lại, Neo4j local có 143 khái niệm và 86 quan hệ, phủ đủ 46 chủ đề. Kiểm thử so sánh toàn bộ bộ ba, trích đoạn và URL với dữ liệu nguồn; nạp lần thứ hai không tăng số cạnh. Kết quả lưu tại `results/graph_coverage.json`.

4.7. Từ nguồn tới kết quả hiển thị

Đường đi phân loại là ZIP/CSV, chọn cột, chia dữ liệu, fit pipeline, lưu checkpoint, API và giao diện. Đường đi hồi quy là hàm tải của thư viện, chia dữ liệu, fit pipeline, checkpoint và API. Đường đi tri thức là đoạn văn có nguồn, chỉ mục TF-IDF và bộ ba Neo4j, lựa chọn bằng chứng rồi tổng hợp trích đoạn.

Các tệp `results/model_metrics.json`, `results/classification_predictions.csv`, `results/regression_predictions.csv` và `results/api_scenarios.json` lần lượt lưu số liệu tổng hợp, dự đoán chi tiết và kiểm thử chức năng. Script `scripts/audit_data.py` sinh bảng dữ liệu của báo cáo và bản danh mục dùng trên web từ cùng một bản chụp dữ liệu. Tách thống kê dữ liệu, metric mô hình và kiểm thử API giúp tránh coi việc ứng dụng chạy thành công là bằng chứng mô hình dự đoán tốt.

Chương 5

Các thành phần và thực nghiệm

5.1. Phân loại

Dữ liệu tổng hợp gồm 50.000 dòng. Tám biến đầu vào là tuổi, BMI, HbA1c, đường huyết lúc đói, glucose, huyết áp tâm thu, HDL và LDL. Các biến chứa điểm nguy cơ, khuyến nghị sức khỏe và yêu cầu gặp bác sĩ không được đưa vào mô hình. Phép chia phân tầng với seed 254 giữ 40.000 dòng huấn luyện và 10.000 dòng kiểm tra.

MLP có ba tầng ẩn 64, 32 và 16 đơn vị, dừng sớm và tối đa 150 vòng lặp. Trung vị và tham số chuẩn hóa được học trên tập huấn luyện ngoài. Cơ chế early stopping của thư viện chia tiếp một phần dữ liệu cho validation; do tiền xử lý nằm trước MLP, các tham số tiền xử lý đã nhìn thấy phần validation nội bộ này. Tập test ngoài vẫn độc lập, nhưng để kiểm soát validation nghiêm ngặt hơn cần tách validation trước toàn bộ pipeline.

Logistic Regression dùng trọng số lớp cân bằng làm đối chứng. MLP dùng cấu hình mặc định không cân bằng trọng số. Vì hai cấu hình có cách xử lý mất cân bằng khác nhau, chênh lệch kết quả không chỉ phản ánh kiến trúc mạng. Accuracy cần đọc cùng macro-F1, đặc biệt với lớp Low hiếm.

5.2. Hồi quy

Bộ diabetes từ scikit-learn được tải ở dạng chưa chuẩn hóa, gồm 442 quan sát với 10 biến đầu vào. Phép chia seed 254 giữ 353 mẫu huấn luyện và 89 mẫu kiểm tra. Ridge với hệ số điều chuẩn 10 là đối chứng. MLP có ba tầng ẩn 64, 32 và 16, dừng sớm và tối đa 1.000 vòng lặp.

Các biến s1 đến s6 là những trường nghiên cứu của bộ dữ liệu. Không được thay tên hoặc đơn vị bằng suy đoán khi nhập hồ sơ ngoài đời. Mục tiêu của demo là minh họa quy trình suy luận từ mô hình đã lưu. Chất lượng trên bộ dữ liệu nhỏ không chứng minh khả năng chuyển sang quần thể Việt Nam.

5.3. Kết quả kiểm tra

MLP cải thiện accuracy và macro-F1 so với Logistic Regression trong lần chạy này. Tuy nhiên macro-F1 còn thấp so với accuracy, phản ánh hiệu năng không đồng đều giữa các lớp. Ở hồi

Bảng 5.1: Phân loại trên 10.000 mẫu kiểm tra.

Mô hình	Accuracy	Macro-F1
Logistic Regression	0.6965	0.5005
MLP	0.8180	0.5441

Bảng 5.2: Hồi quy trên 89 mẫu kiểm tra.

Mô hình	MAE	R^2
Ridge	44.0555	0.4791
MLP	44.3225	0.4758

quy, Ridge có MAE thấp hơn và hệ số xác định cao hơn MLP. Báo cáo vẫn trình diễn MLP để minh họa nhánh Deep Learning, đồng thời công bố đầy đủ đối chứng và không nhận MLP là mô hình tốt nhất.

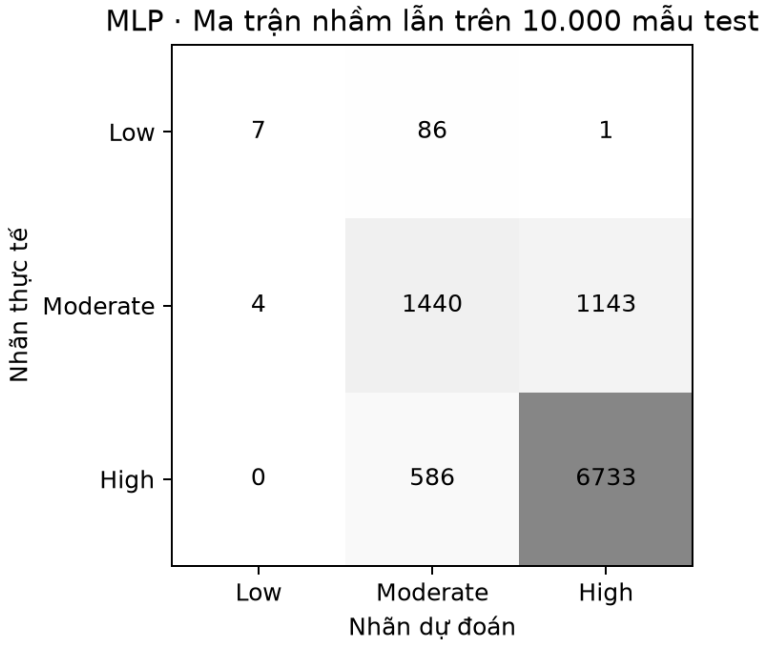
5.4. Truy xuất và đồ thị

Văn bản được chuẩn hóa không dấu rồi biểu diễn bằng TF–IDF từ và ký tự. Điểm kết hợp dùng trọng số 0,55 và 0,45, có cộng ưu tiên nhỏ cho từ khóa và chủ đề. Câu hỏi được biến đổi bằng cùng từ vựng. Điểm truy xuất là tích vô hướng của các vector đã chuẩn hóa. Hệ thống giữ tối đa bốn đoạn có điểm đạt ngưỡng 0,15 và ít nhất 58% điểm của đoạn đứng đầu. Các ngưỡng là cấu hình triển khai, chưa được tối ưu trên một bộ validation độc lập.

Agent đồ thị lấy các cạnh có thuộc tính document trùng với mã tài liệu đã truy xuất, kiểm tra bộ ba, trích đoạn và URL rồi cấp quan hệ có nguồn vào ngữ cảnh Qwen. Đây là truy xuất văn bản kết hợp căn cứ đồ thị theo tài liệu, chưa phải bộ máy suy luận nhiều bước. Một nhãn quan hệ không chứng minh quan hệ nhân quả. Khi không có tài liệu đủ điểm, hệ thống yêu cầu làm rõ; khi Neo4j mất kết nối, nó dùng văn bản KB và ghi rõ nhánh lỗi.

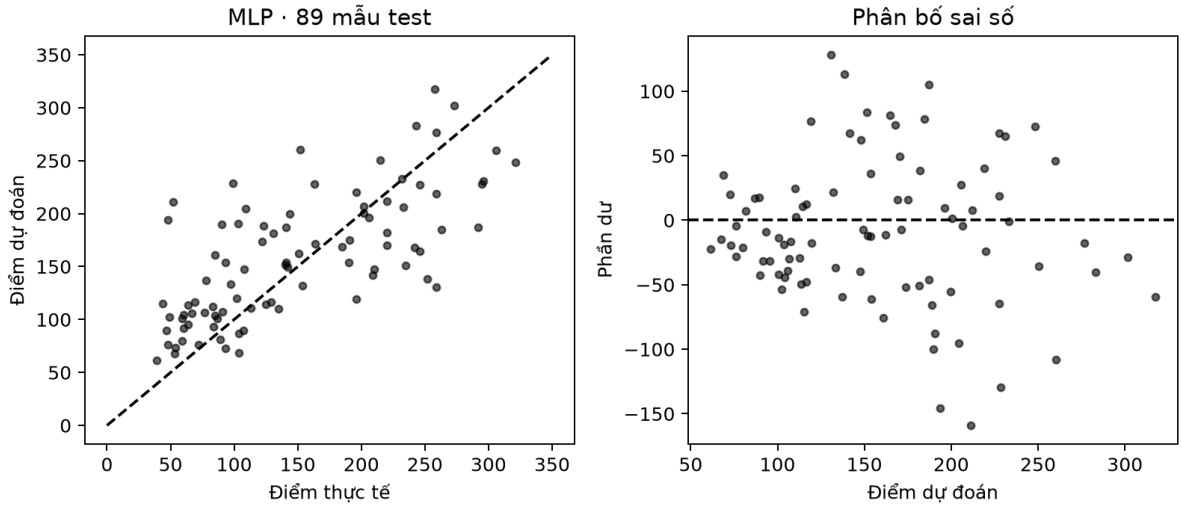
5.5. Phân tích sai số theo lớp

Ma trận nhầm lẫn được tạo từ toàn bộ 10.000 dự đoán ngoài mẫu, không lấy từ tập huấn luyện. Tập test có 7.319 mẫu High, 2.587 mẫu Moderate và 94 mẫu Low. MLP dự đoán đúng 6.733 mẫu High, 1.440 mẫu Moderate nhưng chỉ 7 mẫu Low. Vì vậy accuracy 81,8% che giấu khả năng nhận diện lớp hiếm còn yếu. Recall của Low chỉ khoảng 7,45%; kết quả này cho thấy hạn chế đáng kể ở lớp hiếm. Tỷ lệ lỗi giữa Moderate và High cho thấy ranh giới của nhãn tổng hợp chưa được mô hình tách tốt. Cải thiện tiếp theo cần so sánh trọng số lớp, điều chỉnh ngưỡng và cách lấy mẫu trên validation, đồng thời giữ nguyên test để tránh chọn cấu hình theo kết quả test. Không đổi tên nhãn tổng hợp thành chẩn đoán y tế.



Hình 5.1: Ma trận nhầm lẫn của MLP trên tập kiểm tra phân tầng.

5.6. Phân tích phần dư hồi quy



Hình 5.2: Dự đoán và phần dư của MLP trên 89 mẫu kiểm tra.

Đường chéo thể hiện dự đoán bằng giá trị thật. Những điểm cách xa đường chéo đóng góp lớn vào MAE. Phần dư được tính bằng giá trị thực trừ dự đoán; phần dư dương biểu thị dự đoán thấp hơn thực tế. Không thể dùng một chỉ số tổng hợp để bảo đảm mọi nhóm đầu vào có sai số tương đương. Bản này chưa xây dựng khoảng dự báo và chưa hiệu chỉnh bất định.

5.7. Hợp đồng dữ liệu và API

GET /api/schema cung cấp thứ tự đặc trưng. GET /api/examples cung cấp một mẫu từ mỗi bộ dữ liệu để trình diễn. POST /api/chat nhận question, task và values. Với task knowledge, hệ thống truy xuất văn bản; với classification hoặc regression, values phải đủ các trường tương ứng. Kết quả gồm answer, result, safety, sources, graph_evidence và trace. Nhánh kiến thức còn có graph_status và graph_context_supplied để phân biệt đồ thị khả dụng với ngữ cảnh thực sự được cấp cho đường sinh được chấp nhận. Trường safety tách mức ưu tiên khỏi cờ rủi ro thuốc; nhánh ưu tiên có thể trả trước khi suy luận mô hình.

Trường result của phân loại gồm label, scores và scope. scores là đầu ra xác suất lớp của mô hình trên nhãn tổng hợp, chưa được hiệu chỉnh. Trường result của hồi quy gồm value và scope. Việc tách cấu trúc này tránh dùng chung một con số cho hai ý nghĩa khác nhau. Dữ liệu mẫu là mẫu của bộ nghiên cứu, không phải hồ sơ do người dùng cung cấp.

Nút "Giải thích kết quả" trên web gửi lại task và values của mẫu vừa chạy, nên câu trả lời được tính từ chính các đầu vào đó. Nhánh phân loại nêu nhãn, điểm của ba lớp, accuracy và macro-F1 trên tập test; nhánh hồi quy nêu điểm tiến triển, MAE và R^2. Mỗi nhánh đều giải thích ý nghĩa của nhãn/điểm và giới hạn khi chuyển sang trường hợp cá nhân. Mã tạo diễn giải nằm trong `app/model_explanation.py`; nó không sinh khuyến nghị điều trị.

Kiểm tra đầu vào hiện yêu cầu số hữu hạn, giới hạn trị tuyệt đối và đủ số trường. Đây là kiểm tra kỹ thuật, chưa phải kiểm định lâm sàng từng đơn vị. Giá trị nhập bằng số nhưng sai đơn vị vẫn có thể làm kết quả vô nghĩa; vì vậy demo được giới hạn ở dữ liệu nghiên cứu và có nút điền mẫu.

5.8. Kết quả kiểm thử tích hợp

Script test_scenarios.py thực hiện tám trường hợp: câu hỏi có nguồn; câu hỏi ngoài từ vựng; suy luận phân loại; suy luận hồi quy; thiếu trường; giá trị không hữu hạn; tên bài toán không hợp lệ; lỗi Neo4j có kiểm soát. Tám trường hợp đều đạt điều kiện kiểm tra ở lần chạy lưu trong results/api_scenarios.json. Kiểm tra lỗi cơ sở dữ liệu được thực hiện bằng thay thế driver trong test, không phải chủ động tắt cơ sở dữ liệu đang phục vụ người dùng.

Kiểm thử xác nhận hợp đồng phản hồi và nhánh lỗi, chưa đo độ chính xác truy xuất trên tập câu hỏi lớn. Trường hợp ngoài miền sử dụng chuỗi không có trong từ vựng, do đó không chứng minh hệ thống phát hiện mọi câu hỏi ngoài miền bằng tiếng Việt. Những giới hạn này giúp tách kiểm thử chức năng khỏi đánh giá chất lượng tư vấn.

Bản cập nhật thêm bảy điều kiện trong `scripts/test_chat_upgrade.py`: bốn chủ đề RAG mới, hai cách diễn giải kết quả mô hình và tổng số tài liệu công bố qua API. Thử nghiệm xác nhận các đoạn dẫn đúng nguồn và không biến điểm nghiên cứu thành chẩn đoán. Đây vẫn

là kiểm tra chức năng theo kịch bản, chưa phải đánh giá y khoa bởi chuyên gia.

5.9. Đối chiếu với sơ đồ yêu cầu

Thành phần	Hiện thực trong sản phẩm
DB và Deep Learning	Hai bộ dữ liệu tách biệt, MLPClassifier và MLPRegressor; có checkpoint và kết quả test.
Phân loại và dự đoán	Hai lựa chọn trong phòng thực nghiệm, cùng chuyển kết quả đến phản hồi và nhật ký.
Web/sách đến KB	Kế thừa KB đã biên soạn từ tiểu luận 02; bảo toàn mã nguồn tài liệu, không tự động thu thập web ở thời điểm chạy.
KB và RAG	TF-IDF chọn đoạn; agent đồ thị đối chiếu quan hệ có nguồn; Qwen local nhận văn bản và KG trong cùng ngữ cảnh.
Agent và RL + rule	Handler và MessageBus xử lý yêu cầu; chính sách học một bước chọn kiểu phản hồi, luật chặn thiếu bằng chứng.
Tư vấn, trợ giúp	Chatbot giải thích kiến thức và phạm vi đầu ra; bản local dùng Qwen3 4B cho câu kiến thức có nguồn, còn câu thông số và thuốc theo luật.

Bộ nộp riêng trong `exports/diabetes/` cung cấp KB JSON, KG CSV/JSON/Cypher, chỉ mục RAG đã fit, cấu hình, câu hỏi kiểm tra và code chạy lại. Các tệp này bổ sung khả năng kiểm tra dữ liệu và giao thức bên cạnh việc xem giao diện.

Chương 6

KB, KG và RAG trong bộ sản phẩm nộp

6.1. Các thành phần có thể kiểm tra độc lập

GlucoGuide lưu phần tri thức ở dạng dữ liệu và code, bên cạnh giao diện minh họa. Kho KB cung cấp nội dung và nguồn tham khảo; KG biểu diễn các quan hệ có hướng; RAG chọn căn cứ phù hợp cho từng câu hỏi. Ba phần dùng chung mã tài liệu để có thể truy từ một câu trả lời về đoạn văn và quan hệ đã sử dụng.

Bảng 6.1: Các tệp chính trong bộ nộp tri thức.

Phần	Tệp	Nội dung
KB	`kb/knowledge.json`	46 đoạn, 19 nguồn, từ khóa, văn bản và bộ ba.
KG	`kg/nodes.csv`, `kg/relationships.csv`	143 nút, 86 cạnh, mã tài liệu, trích đoạn và URL nguồn.
KG	`kg/graph.json`, `kg/import.cypher`	Bản chụp Neo4j đã đối chiếu và câu lệnh MERGE để nạp lại.
RAG	`rag/index.joblib`, `rag/settings.json`	Chỉ mục từ/ký tự đã fit và các ngưỡng đang sử dụng.
Đánh giá	`rag/queries.json`, `rag/evaluation.json`	Câu hỏi, tài liệu kỳ vọng, tài liệu được chọn và chỉ số tổng hợp.
Agent	`examples/agent_runs.json`, `code/`	Ví dụ chạy local, thông điệp giữa các agent và code truy vấn offline.
Toàn vẹn	`manifest.json`	Số lượng, kích thước và SHA-256 của từng tệp.

Các tệp nằm trong `exports/diabetes/`; bản ZIP tương ứng là `exports/glucoguide_kb_kg_rag.zip`. Trọng số Qwen, mật khẩu Neo4j và trạng thái hội thoại cá nhân không được đưa vào gói. Phần code kèm theo đủ để truy vấn KB và bản chụp KG mà không cần mở web.

Bộ tệp tri thức trên repository
https://github.com/jellalaz/He_Thong_Thong_Minh_Thay_Que/tree/main/exports/diabetes

6.2. Agent đồ thị tham gia luồng trả lời

6.2.1. Trách nhiệm và hợp đồng thông điệp

Agent Truy xuất nhận kế hoạch của lượt hỏi và trả tối đa bốn đoạn văn. Điều phối gửi danh sách mã tài liệu đó đến agent Đồ thị tri thức qua MessageBus. Handler của agent thực hiện truy vấn Neo4j và trả các trường: khái niệm đầu, quan hệ, khái niệm cuối, mã tài liệu, trích đoạn và URL nguồn. Nhật ký ghi REQUEST, INFORM hoặc FAILURE từ chính lần gọi handler.

Truy vấn sử dụng tham số `ids`; câu hỏi người dùng không được ghép vào mã Cypher. Bộ đọc lấy tối đa 48 quan hệ ứng viên và chọn tối đa 12 quan hệ cho ngữ cảnh. Khi nhiều tài liệu cùng có căn cứ, các quan hệ được lấy luân phiên theo tài liệu để một đoạn không chiếm toàn bộ phần đồ thị.

```
MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept)
WHERE r.document IN $ids
RETURN a.name AS source, r.label AS relation,
       b.name AS target, r.document AS document,
       r.evidence AS evidence, r.source_url AS source_url
ORDER BY document, source, relation, target LIMIT 48
```

6.2.2. Đối chiếu căn cứ trước khi đưa vào mô hình

Một cạnh chỉ được chấp nhận khi mã tài liệu thuộc nhóm đã truy xuất, bộ ba khớp dữ liệu KB, trích đoạn khớp văn bản và URL khớp nguồn của đoạn đó. Quan hệ trùng bị bỏ. Cạnh không khớp không được đưa vào prompt, kể cả khi nó tồn tại trong cơ sở dữ liệu.

Những cạnh đã đạt kiểm tra được trình bày cùng văn bản dưới dạng "khái niệm đầu → quan hệ → khái niệm cuối", kèm mã tài liệu. Qwen nhận cả hai dạng căn cứ trong cùng yêu cầu local. Bước kiểm chứng phản hồi tiếp tục đối chiếu mã nguồn, số liệu và phạm vi; nó không phải một cơ chế chứng minh mọi suy luận của mô hình đều đúng.

Trường `graph_context_supplied` cho biết phản hồi Qwen được chấp nhận có ngữ cảnh đồ thị hay không. Trường này ghi nhận dữ liệu đã cấp vào đường sinh, không đo mức độ mô hình thực sự dựa vào từng cạnh. Các cạnh hiển thị cùng phản hồi được giới hạn theo những tài liệu mà bản sinh đã dẫn mã.

6.2.3. Mất kết nối và thứ tự ưu tiên

Khi Neo4j không hoạt động, MessageBus ghi FAILURE của agent đồ thị. Hệ thống tiếp tục bằng văn bản KB, để trạng thái đồ thị là unavailable và không tạo các quan hệ thay thế. Bản truy vấn offline dùng dữ liệu đã xuất chỉ khi người chạy chủ động chọn công cụ đi kèm; đầu ra ghi rõ exported-snapshot.

Nhánh cấp cứu được phân loại trước toàn bộ bước truy xuất. Nó không chờ KG, chỉ mục RAG hoặc Qwen. Do đó việc bổ sung đồ thị không thay đổi thứ tự xử lý các tình huống cần hành động ưu tiên.

6.3. Xuất và chạy lại bộ nộp

6.3.1. Kiểm tra nguồn trước khi xuất

Script `scripts/export_knowledge.py` chỉ kết nối Neo4j trên loopback. Nó đọc toàn bộ nút và cạnh Concept, đối chiếu chính xác tập bộ ba với KB, rồi kiểm tra trích đoạn và URL. Nếu hai nguồn không khớp, thao tác xuất dừng thay vì ghi một bản chụp thiếu hoặc tự dựng dữ liệu từ hình ảnh.

CSV dùng UTF-8 có BOM để giữ chữ tiếng Việt khi mở bằng các công cụ bảng tính trên Windows. Định danh n0001, n0002 và các giá trị tiếp theo là mã của bộ xuất, không phải ID nội bộ Neo4j. JSON bảo toàn tên khái niệm, quan hệ, tài liệu và bằng chứng; Cypher dùng MERGE nên có thể nạp các dữ liệu tương ứng mà không cần xóa cơ sở dữ liệu đang có.

6.3.2. Chỉ mục và cấu hình RAG

Chỉ mục đã fit chứa hai bộ vector hóa, ma trận tài liệu và danh mục đoạn văn. Cấu hình đi kèm ghi n-gram từ 1–2, n-gram ký tự 3–5, trọng số 0,55/0,45, ngưỡng điểm 0,15, tỷ lệ so với đoạn đầu 0,58 và tối đa bốn kết quả. Code của bộ truy xuất được lưu cùng gói để đọc lại đối tượng joblib.

Checksum được kiểm tra trước khi nạp chỉ mục. Joblib vẫn là định dạng chỉ nên mở từ nguồn tin cậy; SHA-256 giúp phát hiện tệp bị thay đổi, không xác thực danh tính người phát hành. Phiên bản thư viện được ghi trong requirements của bộ nộp để làm rõ điều kiện tái lập.

6.3.3. Lệnh sử dụng

Từ project gốc, xuất dữ liệu và lấy các ví dụ hội thoại trên máy:

```
python -m scripts.export_knowledge --live-examples
python -m scripts.test_knowledge_export
```

Từ thư mục `exports/diabetes/`, truy vấn bộ nộp không cần web hoặc kết nối mạng:

```
python code/scripts/query_knowledge_export.py . --question "HbA1c là gì?"
```

Lệnh này chạy các agent truy xuất, đồ thị bản chụp và tổng hợp căn cứ. Nó không chạy Qwen hoặc giả lập một phản hồi ngôn ngữ sinh. Muốn trình diễn hội thoại đầy đủ, dùng ứng dụng local trong project gốc. Để nạp lại vào Neo4j local, dùng `code/scripts/restore_knowledge_export.py`; thông tin đăng nhập được nhập riêng, không lưu trong gói.

6.4. Kết quả kiểm tra

Bảng 6.2: Kết quả truy xuất và kiểm tra bộ nộp.

Hạng mục	Kết quả
Số tệp trong bộ nộp	26
Câu hỏi truy xuất	22
Hit@1	90.0%
Hit@4	100.0%
MRR@4	0.950
Kiểm tra agent đạt	11
Kiểm tra bộ nộp đạt	8

Tập truy xuất gồm 20 câu có tài liệu kỳ vọng và hai câu cần không trả tài liệu. Câu hỏi bao phủ kiến thức bệnh, bữa ăn, thuốc ở mức thông tin, vận động, theo dõi, câu không dấu và một lượt hỏi tiếp. Danh sách tài liệu kỳ vọng được ghi trước khi chạy phép đo. Đây là tập kiểm tra chức năng do project biên soạn, chưa phải một benchmark y khoa độc lập.

Hit@1 đo tỷ lệ câu có tài liệu kỳ vọng ở vị trí đầu. Hit@4 đo tỷ lệ tìm thấy ít nhất một tài liệu kỳ vọng trong danh sách trả về; danh sách có thể ngắn hơn bốn đoạn. MRR@4 là trung bình nghịch đảo thứ hạng đúng đầu tiên. Các chỉ số này đánh giá bước truy xuất, không phải độ chính xác chẩn đoán hay mức an toàn của toàn bộ câu trả lời.

Bộ kiểm tra gói nộp phát lại 22 truy vấn từ chỉ mục đã lưu khi socket bị vô hiệu hóa; đối chiếu CSV, JSON và ZIP; chạy entry point bằng các module đã đóng gói; phát hiện tệp bị sửa; và dùng EXPLAIN để kiểm tra Cypher mà không thực hiện ghi dữ liệu. Các kiểm tra agent riêng xét nguồn sai, bộ ba sai, mã tài liệu lạ, bản ghi trùng, việc cấp KG vào prompt, mất Neo4j và nhánh cấp cứu.

Ba ví dụ thông thường chạy trên Qwen local đều trả phản hồi được chấp nhận với hai quan hệ có nguồn. Ví dụ đường huyết cao kèm nôn, đau bụng đi theo luật cấp cứu và không gọi bước sinh. Các phản hồi và trace nguyên văn nằm trong `examples/agent_runs.json`; số ví dụ nhỏ này chỉ dùng để đối chiếu luồng thực thi.

6.5. Phạm vi và giới hạn

KG hiện còn 57 cụm rời nhau và nhiều cụm chỉ có hai hoặc ba khái niệm. Hệ thống truy vấn quan hệ theo tài liệu đã chọn, chưa tìm đường nhiều bước, chưa hợp nhất một ontology y khoa và chưa triển khai thuật toán cộng đồng của Microsoft GraphRAG. Mở rộng các bước đó cần thêm bài toán đánh giá cụ thể, không thể chỉ dựa vào việc đồ thị có nhiều nút hơn.

Các agent cùng chạy trong một tiến trình dưới điều phối tập trung. Cách tổ chức cho phép quan sát giao thức và kiểm thử từng vai trò, nhưng chưa chứng minh lợi ích của các agent tự chủ độc lập. Chất lượng nội dung vẫn phụ thuộc nguồn, độ phủ KB, cách biên soạn bộ ba và kiểm tra đầu ra. Trước khi áp dụng ngoài thực nghiệm học phần cần có đánh giá chuyên môn và dữ liệu phù hợp.

Chương 7

Code và demo

7.1. Địa chỉ và cách chạy

Bản hoàn thiện chạy trên máy cá nhân

`http://localhost:8765`

Xem đồ thị trên web, không cần nhập lệnh

`http://localhost:8765/#graph`

Neo4j Browser cục bộ

`http://localhost:7474/browser/`

Kho mã nguồn

`https://github.com/jellalaz/He_Thong_Thong_Minh_Thay_Que`

Demo Vercel của phiên bản trước

`https://glucoguide-giang254.vercel.app`

Các địa chỉ localhost chỉ truy cập dịch vụ trên máy đang chạy sản phẩm. Bản hoàn thiện dùng Qwen3 4B và Neo4j local; không thể mở từ máy khác bằng các địa chỉ này. Bản Vercel là triển khai trước đây, chưa được xác nhận đồng bộ mô hình, đồ thị và luật an toàn mới.

Trên máy thực nghiệm, chạy `start-neo4j.ps1` nếu cơ sở dữ liệu chưa hoạt động, rồi mở `START_CHATBOT_LOCAL.cmd` để khởi động Qwen và web. Tệp requirements.txt mô tả thư viện ứng dụng; requirements-training.txt dành cho tái lập huấn luyện. Các script `scripts/train_models.py`, `scripts/train_policy.py` và `scripts/ingest_graph.py` lần lượt tạo checkpoint, chính sách và dữ liệu đồ thị. Quá trình phục vụ dùng tài nguyên đã có trên máy, không tự tải mô hình hoặc tài liệu. Cấu hình Neo4j nằm trong .env và không đưa lên repository.

7.2. Kịch bản trình diễn

Kịch bản thứ nhất: hỏi "HbA1c là gì?". Kiểm tra nội dung, mở nguồn tham khảo, đối chiếu bước truy xuất và số quan hệ Neo4j. Kịch bản thứ hai: hỏi một chủ đề không có trong kho,

chẳng hạn lịch sử một đội bóng. Hệ thống phải nêu phạm vi hỗ trợ hoặc hỏi làm rõ, không tự tạo nguồn.

Kịch bản thứ ba: nhập đủ tám trường phân loại bằng một mẫu của dữ liệu. Ghi lại nhãn, thông báo phạm vi và nhật ký mô hình. Kịch bản thứ tư: nhập đủ mười trường hồi quy từ mẫu nghiên cứu, kiểm tra đầu ra số và thông báo đây là điểm tiến triển. Hai kịch bản không dùng chung biểu mẫu.

Kịch bản thứ năm: bỏ một trường bắt buộc. Hệ thống phải nêu trường còn thiếu. Kịch bản thứ sáu: nhập giá trị không hữu hạn hoặc chuỗi ký tự vào API, hệ thống phải từ chối. Kịch bản thứ bảy: dùng kiểm thử giả lập Neo4j mất kết nối để kiểm tra nhánh dự phòng; không cần tắt dịch vụ đang phục vụ người dùng. Kịch bản thứ tám: mở Neo4j Browser và thực hiện truy vấn dưới đây để đối chiếu với giao diện đồ thị.

```
MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept)
RETURN a,r,b LIMIT 100;
```

7.3. Hạn chế và hướng hoàn thiện

Hệ thống duy trì ngữ cảnh chủ đề trong phiên nhưng chưa có bộ nhớ hội thoại bền vững, chưa đọc phiếu xét nghiệm, nhánh Qwen local mới được đánh giá trên tập nội bộ nhỏ và chưa có hội đồng tranh biện độc lập. Kiểm chứng mô hình mới chỉ dùng một seed; chưa có đánh giá chuyên gia, chưa có dữ liệu ngoài miền và chưa kiểm thử tải lớn. RL học quy trình mô phỏng, không quyết định điều trị.

Hướng ưu tiên là mở rộng tập câu hỏi độc lập, kiểm tra đơn vị có nguồn và đánh giá vai trò của KG bằng phép loại bỏ thành phần trên cùng tập câu hỏi. Chất lượng nguồn và khả năng phát hiện thiếu căn cứ cần được đo cùng độ đúng chủ đề và thời gian trả lời.

7.4. Bản triển khai công khai và bản local

Bản công khai đã triển khai
https://glucoguide-giang254.vercel.app

Bản Vercel Hobby và Neo4j AuraDB Free trình diễn giao diện, dữ liệu bảng và truy xuất của phiên bản trước. Kết quả triển khai lưu tại `results/deployment_checks.json`. Báo cáo đánh giá bản local với Qwen3 4B, kho tri thức đầy đủ và bộ luật an toàn đã bổ sung.

Qwen3 4B chạy trên máy cá nhân. Đồ thị local có 46 chủ đề, 143 khái niệm và 86 quan hệ; bản công khai chưa được xác nhận đồng bộ những thay đổi này. Kịch bản trình diễn đầy đủ sử dụng địa chỉ local cùng mô hình và cơ sở dữ liệu đang chạy.

Chương 8

Hội thoại cục bộ và thực nghiệm mô hình ngôn ngữ

8.1. Mô hình dùng trong bản triển khai

GlucoGuide sử dụng Qwen3 4B ở định dạng GGUF, lượng tử hóa Q4_K_M, để diễn đạt câu trả lời từ tài liệu truy xuất. Tệp mô hình có dung lượng 2.497.280.256 byte; mã SHA-256 được đối chiếu trước khi đưa vào sử dụng. Bộ chạy llama.cpp dùng Vulkan trên NVIDIA GeForce RTX 4060 Laptop GPU. Máy thực nghiệm có Intel Core i9-14900HX và RAM khoảng 16 GB.

Trọng số Qwen3 4B đã được nhà phát triển huấn luyện sẵn và được giữ nguyên trong bản triển khai này. Phần xây dựng của project gồm kho tri thức, truy xuất, ngữ cảnh hội thoại, giao thức điều phối, luật an toàn và giao diện. Hai MLP dữ liệu bảng và bộ nhận diện ý định được huấn luyện riêng.

Mô hình dùng trong bản local
`https://huggingface.co/Qwen/Qwen3-4B-GGUF`

8.2. Đường đi của một lượt hỏi

8.2.1. Phân loại an toàn và điều phối

Yêu cầu từ web đi qua bộ phân loại an toàn trước khi xét chủ đề hoặc suy luận mô hình. Tình huống cần hành động ưu tiên được trả lời bằng luật nội bộ. Với câu thông thường, lớp hội thoại xác định nhu cầu hiện tại, bổ sung chủ đề của lượt trước khi phù hợp, rồi chuyển truy vấn đến kho tri thức.

Bốn thông điệp gần nhất được giới hạn độ dài trước khi đưa vào mô hình. Việc làm rõ thông số dựa trên dữ kiện người dùng đã cung cấp. Khi đổi chủ đề hoặc chủ thể, hệ thống không tự đem số đo trước đó áp sang câu hỏi mới. Lịch sử này phục vụ phiên đang mở; chưa phải hồ sơ sức khỏe hoặc bộ nhớ bền vững.

8.2.2. Truy xuất và tạo phản hồi

Kho tri thức được tìm bằng TF–IDF từ/cụm hai từ và TF–IDF chuỗi ký tự dài ba đến năm ký tự. Điểm kết hợp dùng trọng số 0,55 và 0,45, sau đó cộng ưu tiên nhỏ cho từ khóa và chủ đề. Hệ

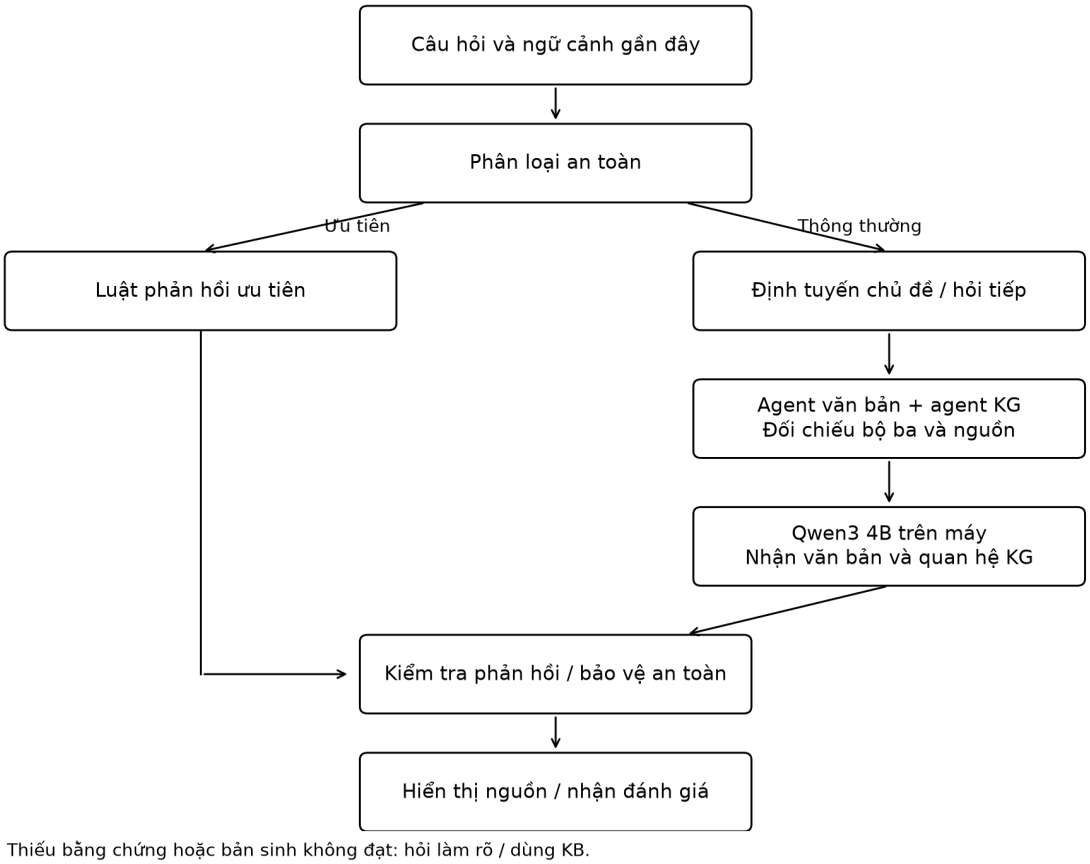
thống giữ tối đa bốn đoạn đạt ngưỡng. Cách kết hợp này hỗ trợ câu không dấu và một số lỗi gõ; chưa sử dụng embedding.

Mô hình nhận chỉ dẫn, tài liệu, quan hệ Neo4j đã đối chiếu, ngữ cảnh gần đây và cách trình bày mong muốn. Mỗi quan hệ giữ mã tài liệu để kiểm tra lại với văn bản; KG không được dùng để suy ra một chẩn đoán hoặc ngưỡng mới. Cấu hình giới hạn 650 token trả lời, temperature 0,2 và seed 254; chế độ suy nghĩ nội bộ được tắt khi phục vụ. llama.cpp được khởi động với cửa sổ ngữ cảnh 8.192 token và một slot. Đây là cấu hình vận hành trên máy thực nghiệm, chưa được tối ưu trên một tập người dùng độc lập.

8.2.3. Đối chiếu và sửa phản hồi

Bộ kiểm tra rà mã tài liệu, số liệu ngoài nguồn, một số cách diễn đạt về liều thuốc, lỗi ngôn ngữ và lỗi phân nhóm thực phẩm đã gặp. Nếu phản hồi chưa đạt, mô hình được yêu cầu viết lại một lần với cùng tài liệu. Khi lần viết lại vẫn không đạt, hệ thống dùng đoạn KB phù hợp.

Mã tài liệu được giữ để đối chiếu nội bộ và bỏ khỏi văn bản trước khi hiển thị. Mục tham khảo cung cấp tên bài, trích đoạn và liên kết gốc. Với phản hồi Qwen, danh sách hiển thị chỉ giữ các đoạn có mã được dùng trong bản sinh. Trích dẫn không tự chứng minh mọi câu đều chính xác; bộ lọc chỉ kiểm tra một số điều kiện nhất quán.



Hình 8.1: Luồng hội thoại local với bước phân loại an toàn trước truy xuất.

8.3. Nhận diện ý định và học cách trình bày

8.3.1. Bộ nhận diện câu hỏi tiếng Việt

TF–IDF ký tự kết hợp hồi quy logistic được huấn luyện trên 112 câu; 32 câu được giữ riêng để kiểm tra. Tám lớp gồm ăn uống, trái cây, vận động, thuốc, theo dõi, huyết áp, HbA1c và ngoài phạm vi. Accuracy nội bộ đạt 93,75%, macro-F1 đạt 0,937. Mỗi câu, nhãn và dự đoán được lưu tại `results/intent_evaluation.json`.

Các câu do project biên soạn, có chủ đề gần nhau giữa hai tập. Kết quả phản ánh khả năng định tuyến trong phạm vi khảo sát; chưa đo năng lực hiểu mọi câu tiếng Việt. Luật ưu tiên và kiểm tra thông số được thực hiện trước khi dùng kết quả phân lớp.

8.3.2. Phản hồi theo từng chủ đề

Người dùng chọn trả lời ngắn gọn, hướng dẫn từng bước hoặc giải thích kỹ. Chế độ tự chọn dùng bảng Q lưu riêng cho từng chủ đề. Đánh giá hữu ích cho thưởng $+1$; chưa hữu ích cho thưởng -1. Giá trị của cách trình bày đã chọn cập nhật theo $Q \leftarrow Q + 0,2(r - Q)$.

Bảng Q này điều chỉnh cách trình bày; không thay trọng số Qwen hoặc ngưỡng an toàn. Tệp trạng thái lưu bảng điểm cùng số lượt đánh giá, không ghi nội dung hội thoại. Chính sách điều phối mô phỏng một bước ở phần trước và chính sách phản hồi theo chủ đề có mục tiêu, dữ liệu và phạm vi sử dụng riêng.

8.4. Thực nghiệm tinh chỉnh Qwen3 1.7B

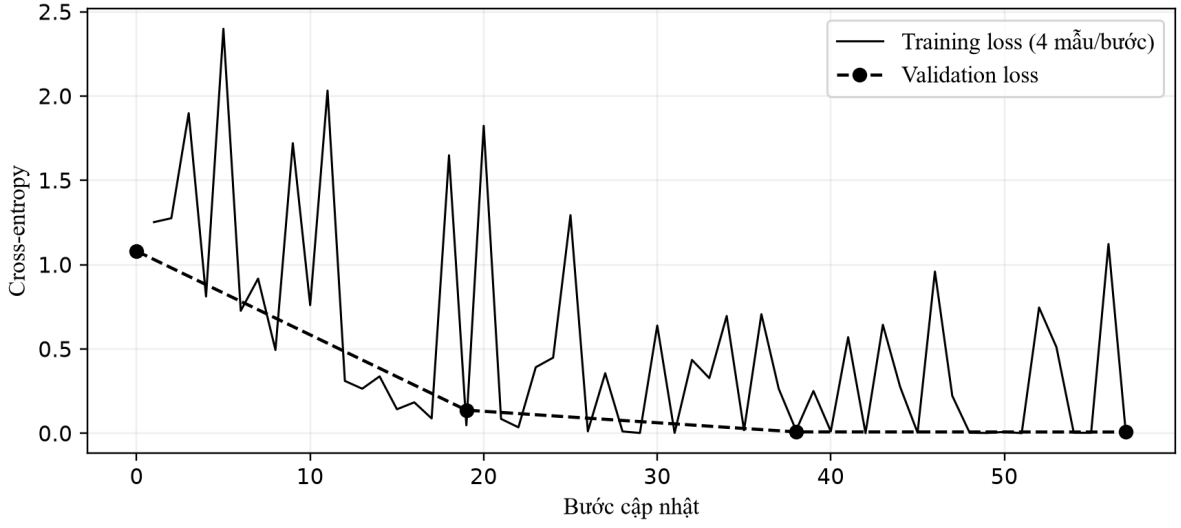
8.4.1. Dữ liệu và cấu hình

Trước khi chọn mô hình 4B cho bản triển khai, project tinh chỉnh LoRA trên Qwen3 1.7B. Bộ dữ liệu có 76 mẫu train, 21 mẫu validation và 16 mẫu test; mỗi mẫu gồm câu hỏi, lịch sử, mã tài liệu và phản hồi đích. Câu hỏi trùng trực tiếp giữa các tập được kiểm tra sau chuẩn hóa. Cùng nguồn KB và cách biên soạn làm tính độc lập của đánh giá còn hạn chế.

Cấu hình dùng LoRA hạng 8, alpha 16, dropout 0,05 trên phép chiếu query và value. Batch size bằng 1, tích lũy gradient qua 4 mẫu, tối đa 768 token/mẫu; AdamW có learning rate 10^{-4} trong 3 epoch. Checkpoint được chọn theo loss validation. Đây là tinh chỉnh adapter, giữ cố định trọng số nền.

8.4.2. Kết quả đo trên máy thực nghiệm

Số tham số được tối ưu là 1,605,632, trên tổng 1,722,180,608 tham số. Checkpoint được chọn ở epoch 3. Loss validation của mô hình nền là 1.0810; giá trị tốt nhất sau tinh chỉnh là 0.0082. Thời gian ba epoch, gồm đánh giá validation và lưu checkpoint, là 1.97 phút. Bộ nhớ GPU cấp phát đỉnh do PyTorch ghi nhận là 3.93 GiB; số này không bao gồm toàn bộ bộ nhớ các ứng dụng khác.



Hình 8.2: Loss huấn luyện theo bước và validation theo epoch. Điểm đầu validation thuộc mô hình nền.

Mô hình	Nhóm	Có dẫn mã	Mã lạ	Chạm giới hạn	Giây/câu
Qwen nền	Kiến thức	0/8	0	0	8.07
Qwen nền	Hội thoại	0/8	0	0	4.97
Qwen LoRA	Kiến thức	8/8	0	0	7.57
Qwen LoRA	Hội thoại	7/8	1	0	4.93

Bảng 8.1: Kiểm tra hình thức đầu ra trên hai nhóm test, mỗi nhóm 8 câu.

Có dẫn mã chỉ kiểm tra sự xuất hiện của mã tài liệu, không chứng minh trích dẫn hỗ trợ đầy đủ câu trả lời. Mã lạ là mã không nằm trong ngữ cảnh đã cấp. Chạm giới hạn nghĩa là sinh đủ 320 token, có nguy cơ câu bị cắt. Độ trễ được đo sau khi mô hình đã nạp, không gồm thời gian khởi động hoặc truy xuất Neo4j. Hai mô hình dùng cùng ngữ cảnh được chỉ định trước để tách ảnh hưởng của bộ sinh; kết quả này chưa đo toàn bộ pipeline truy xuất.

Tệp `results/local_qwen/base_answers.json` và `results/local_qwen/lora_answers.json` lưu nguyên văn từng phản hồi. Tệp `results/local_qwen/training_summary.json` lưu cấu hình, mã băm dữ liệu và tài nguyên đo được. Các tệp này cho phép kiểm tra lại kết luận mà không cần dựa vào ảnh minh họa.

8.4.3. Kết quả và quyết định sử dụng

Adapter giúp phản hồi bám gần đoạn nguồn và định dạng dẫn mã hơn. Tuy nhiên, các lượt hỏi tiếp và thay đổi món ăn còn dễ lặp văn bản cũ. Một lượt tiếp tục huấn luyện trên 46 mẫu train, 13 validation và 16 test làm validation loss giảm từ 0,5873 xuống 0,5201, nhưng phản hồi vẫn có trường hợp sao chép chỉ dẫn hoặc sai sắc thái giao tiếp. Vì vậy các adapter được giữ làm kết quả thực nghiệm; sản phẩm hiện dùng Qwen3 4B kết hợp RAG.

Log, cấu hình và đầu ra nguyên văn nằm tại `results/local_qwen/` và `results/local_qwen_round2/`. Các chỉ số loss này thuộc thí nghiệm 1.7B; không được dùng để mô tả chất lượng mô hình 4B.

8.5. Vận hành và kịch bản trình diễn

GlucoGuide trên máy cá nhân
`http://localhost:8765`

Chạy `START_CHATBOT_LOCAL.cmd` hoặc `run-chatbot-local.ps1`. Web ở cổng 8765, mô hình ở cổng 11435 và Neo4j ở cổng 7687, đều dùng địa chỉ loopback. Bộ gọi mô hình từ chối địa chỉ ngoài; chế độ local cũng từ chối kết nối đồ thị tới máy chủ ngoài. Sau khi có sẵn tài nguyên, luồng hỏi đáp không tự tải mô hình hoặc tài liệu.

Kết quả hiển thị trên web được minh họa tại Hình 10.1 trong chương giao diện.

Chương 9

Phân loại an toàn và kiểm thử

9.1. Mục tiêu và vị trí trong hệ thống

Nhận đúng chủ đề chưa đủ để chọn cách phản hồi. Một câu về đường huyết có thể là câu hỏi kiến thức, yêu cầu làm rõ kết quả hoặc mô tả tình huống đang diễn biến. Bộ phân loại an toàn được đặt trước RAG để xem xét đồng thời triệu chứng, số đo, đơn vị, ngữ cảnh và yêu cầu thay đổi thuốc.

9.2. Mức độ ưu tiên và dữ kiện sử dụng

Mức hoặc cờ	Cách xử lý
EMERGENCY	Đưa hành động cấp cứu lên đầu; không chờ Qwen, Neo4j hoặc thông tin thời điểm đo.
URGENT	Khuyến nghị được đánh giá sớm; nêu cách theo dõi và dấu hiệu cần nâng mức xử trí.
ROUTINE	Tiếp tục định tuyến kiến thức, truy xuất và sinh phản hồi theo nguồn.
MEDICATION_RISK	Cờ độc lập, có thể đi cùng các mức trên; không tính liều, đổi liều hoặc hướng dẫn tự ngừng thuốc.

Số đo có đơn vị được xét riêng để tránh lấy liều insulin, tuổi hoặc LDL làm đường huyết. Quy đổi mmol/L sang mg/dL dùng hệ số xấp xỉ 18. Tổ hợp đường huyết cao kèm nôn hoặc đau bụng được xét trước bảng diễn giải xét nghiệm. Ngưỡng 250 mg/dL là ngưỡng vận hành của bản thử nghiệm, không được dùng như tiêu chí chẩn đoán DKA.

Mất đáp ứng, co giật, khó thở, lú lẫn hoặc không nuốt được là những dấu hiệu chức năng cần ưu tiên. Cụm run tay, vã mồ hôi và chóng mặt trong bối cảnh tiểu đường được xét theo nguy cơ hạ đường huyết. Luật nhận diện một số cách nói gián tiếp như gọi không trả lời hoặc lay không tỉnh.

9.3. Phủ định và ngữ cảnh

Bộ kiểm tra xét phủ định gần cụm triệu chứng và tách các mệnh đề tương phản. Câu hỏi định nghĩa, giả định và lịch sử đã hết triệu chứng được kiểm tra để tránh kích hoạt cấp cứu chỉ vì xuất hiện từ khóa. Khi bổ sung triệu chứng ở lượt sau, số đo gần nhất chỉ được dùng nếu ngữ cảnh còn phù hợp; không chuyển số đo của một người sang người khác.

9.4. Bảo vệ phản hồi

Nhánh ưu tiên dùng các thành phần câu tương ứng với nhóm nguy cơ, không so khớp nguyên văn câu test. Bộ sinh và truy xuất không được ghi đè phản hồi này. Bước kiểm tra cuối khôi phục nội dung an toàn nếu một thành phần phía sau đưa ra văn bản không phù hợp.

Với nghi hạ đường huyết, phản hồi nhắc kiểm tra ngay nếu có máy; hướng dẫn xử trí có điều kiện còn tỉnh táo, nuốt được và nêu khi nào cần tìm trợ giúp. Với đường huyết cao kèm triệu chứng cấp tính, phản hồi cảnh báo khả năng DKA hoặc tình trạng cấp tính khác, không xác nhận bệnh qua chat. Yêu cầu đổi thuốc được chuyển về nhóm điều trị.

9.5. Kiểm thử hoàn toàn nội bộ

Tệp `scripts/test_safety_offline.py` chạy 28 tình huống phân loại và một kiểm tra bảo vệ phản hồi. DNS và kết nối ra ngoài bị chặn; loopback được giữ để dùng các thành phần nội bộ. Nhánh an toàn còn được thử khi Neo4j mất kết nối và khi RAG/Qwen bị thay bằng hàm báo lỗi nếu bị gọi sai thứ tự.

Bảng 9.2: Các nhóm kiểm thử an toàn chính.

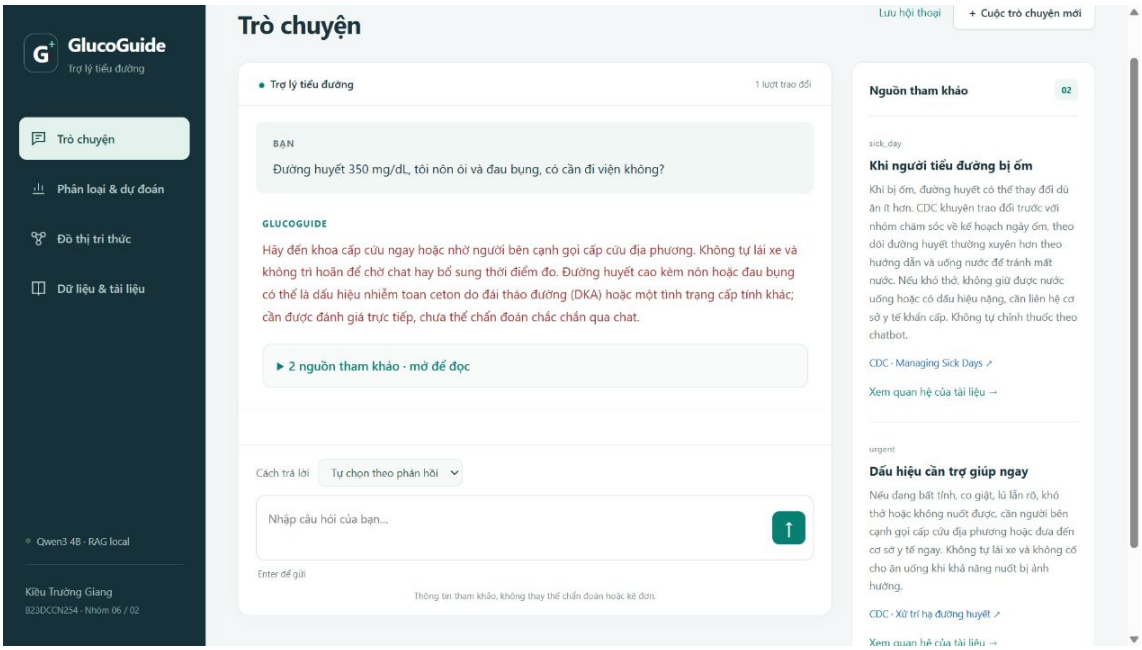
Tình huống	Kết quả đã kiểm tra
Run tay, vã mồ hôi, chóng mặt ở người tiểu đường	URGENT; đo và xử trí có điều kiện.
350 mg/dL kèm nôn, đau bụng	EMERGENCY; không chờ hỏi lúc đói/sau ăn.
300 mg/dL và đề nghị tăng insulin	URGENT, kèm MEDICATION_RISK.
Bất tỉnh hoặc co giật	EMERGENCY; không cho ăn uống qua miệng.
Đường cao, phủ định dấu hiệu cấp cứu	URGENT, không tự nâng thành cấp cứu.

Tình huống	Kết quả đã kiểm tra
Tiểu ra máu	URGENT; không tự kết luận nguyên nhân.
HbA1c 6,2%	ROUTINE; giữ giải thích xét nghiệm.
Đề nghị tự ngừng thuốc	MEDICATION_RISK; không đổi đơn.
Câu hỏi bữa ăn	ROUTINE; giữ luồng tri thức.
Không dấu, sai chính tả, diễn đạt gián tiếp	Nhận diện được các mẫu trong tập test.

Toàn bộ 29 điều kiện an toàn và 81 điều kiện hồi quy đạt trong lần chạy lưu tại `results/safety_offline.json`; số lần thử kết nối ra ngoài bằng không. Hồi quy gồm hội thoại, nguồn tham khảo, chuyển kết quả mô hình sang chat, giao thức agent và bộ lọc phản hồi.

9.6. Giới hạn áp dụng

Phủ định dài, thay đổi thời gian hoặc chủ thể phức tạp có thể vẫn bị hiểu sai. Kết quả xác nhận hành vi phần mềm trên tập câu đã xây dựng, chưa phải đánh giá lâm sàng. Cần rà soát chuyên môn đối với nội dung hướng dẫn, ngưỡng định tuyến và nhóm đặc biệt trước khi mở rộng sử dụng.



Hình 9.1: Nhánh an toàn trên web local: hành động ưu tiên được đưa trước phần diễn giải.

Chương 10

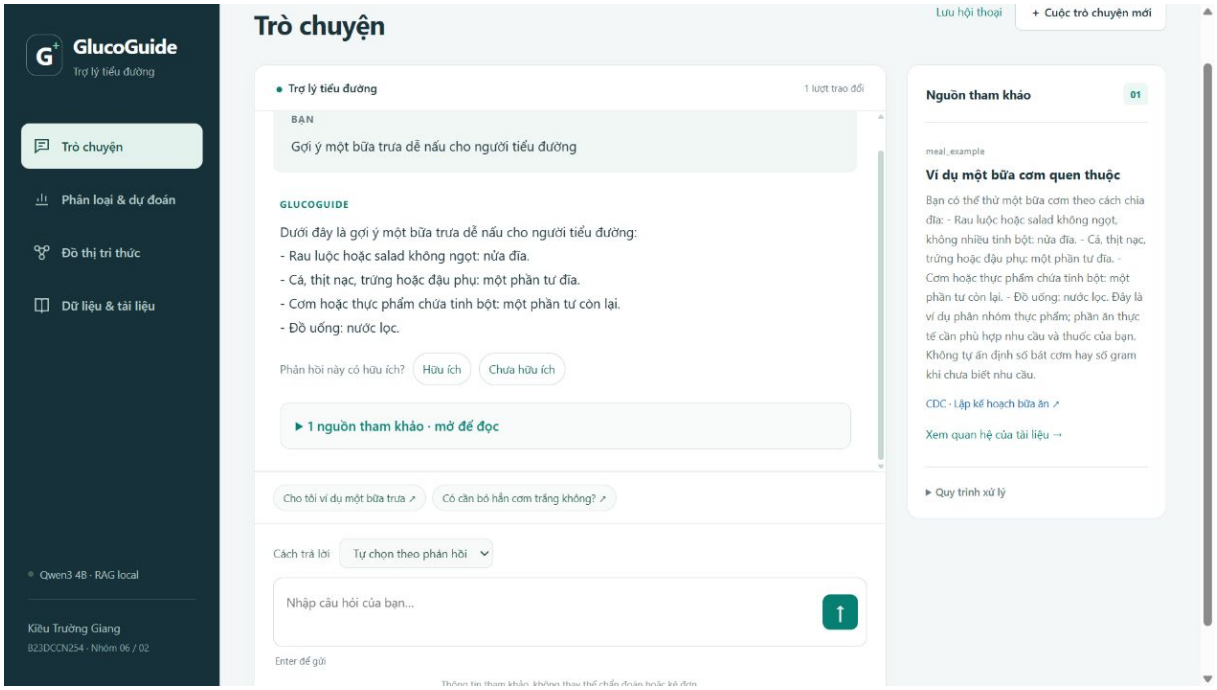
Giao diện và kịch bản sử dụng

10.1. Thiết kế giao diện và cách đọc kết quả

Giao diện có bốn khu vực: Trò chuyện, Phân loại & dự đoán, Đồ thị và Dữ liệu. Mỗi khu vực tương ứng với một nhiệm vụ kiểm tra; khối nhật ký trong cột căn cứ phản ánh lượt đang được chọn. Các hình minh họa ghi lại giao diện và kết quả thực nghiệm trên ứng dụng.

10.1.1. Trợ lý kiến thức

Người dùng nhập câu hỏi hoặc chọn câu hỏi gợi ý. Ví dụ "HbA1c là gì?" được gửi đến POST /api/chat với nhiệm vụ kiến thức. Sau bước phân loại an toàn, bộ truy xuất tìm tối đa bốn đoạn phù hợp. Phản hồi được đối chiếu phạm vi và hiển thị cùng phần nguồn có thể mở để đọc. Mã tài liệu dùng trong xử lý không xuất hiện như ký hiệu rời trong câu trả lời. Nếu không có bằng chứng đạt ngưỡng, hệ thống yêu cầu làm rõ. Giao diện khóa nút gửi trong thời gian chờ và hiển thị lỗi kết nối khi yêu cầu thất bại.



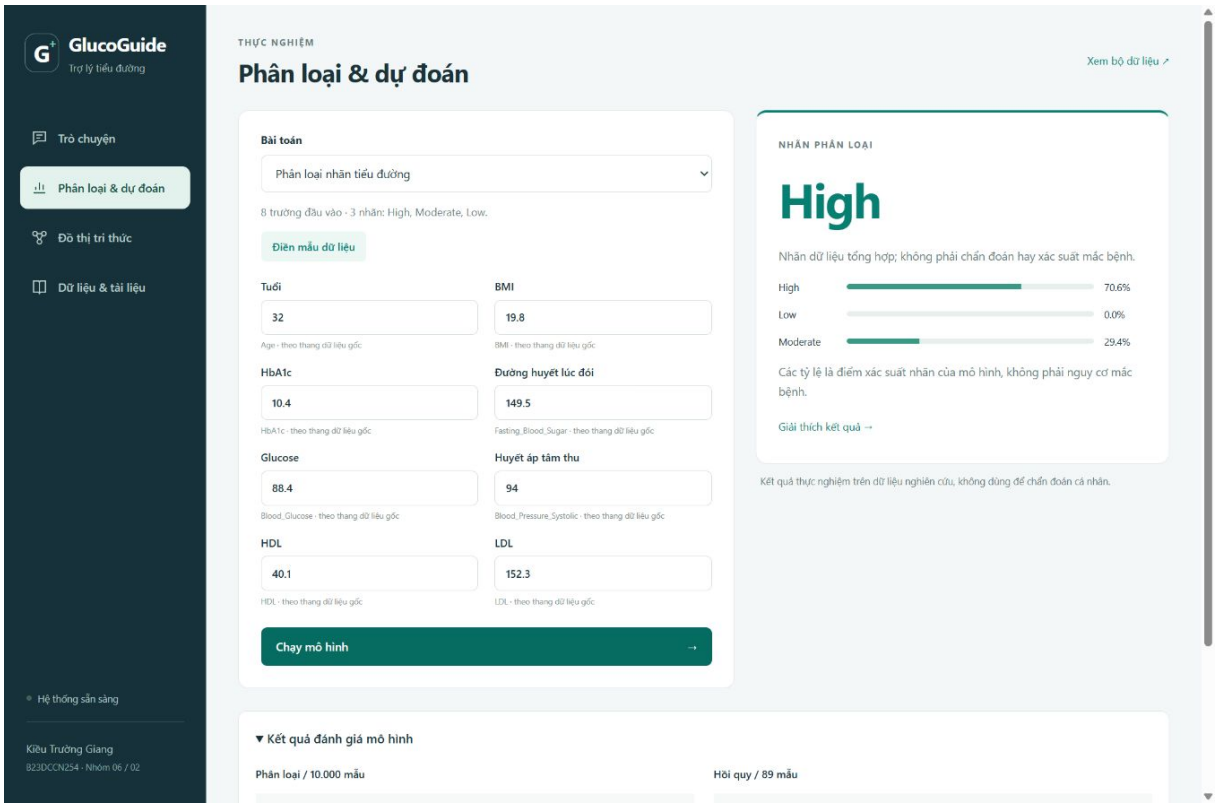
Hình 10.1: Hội thoại trên bản local và cách trình bày nguồn tham khảo.

10.1.2. Thực nghiệm dữ liệu: phân loại

Mở giao diện thực nghiệm

`http://localhost:8765/#models`

Chọn "Phân loại nhãn tiểu đường", bấm "Điền mẫu dữ liệu" rồi chạy mô hình MLP. Biểu mẫu gồm tám trường theo đúng schema của checkpoint. API kiểm tra dữ liệu trước khi pipeline điền thiếu, chuẩn hóa và suy luận. Đầu ra là một trong ba nhãn High, Moderate, Low của bộ dữ liệu tổng hợp. Nhãn này không phải xác suất hay chẩn đoán. Kết quả cần đọc cùng macro-F1 và recall lớp Low trong phần thực nghiệm, bởi tập dữ liệu mất cân bằng mạnh.



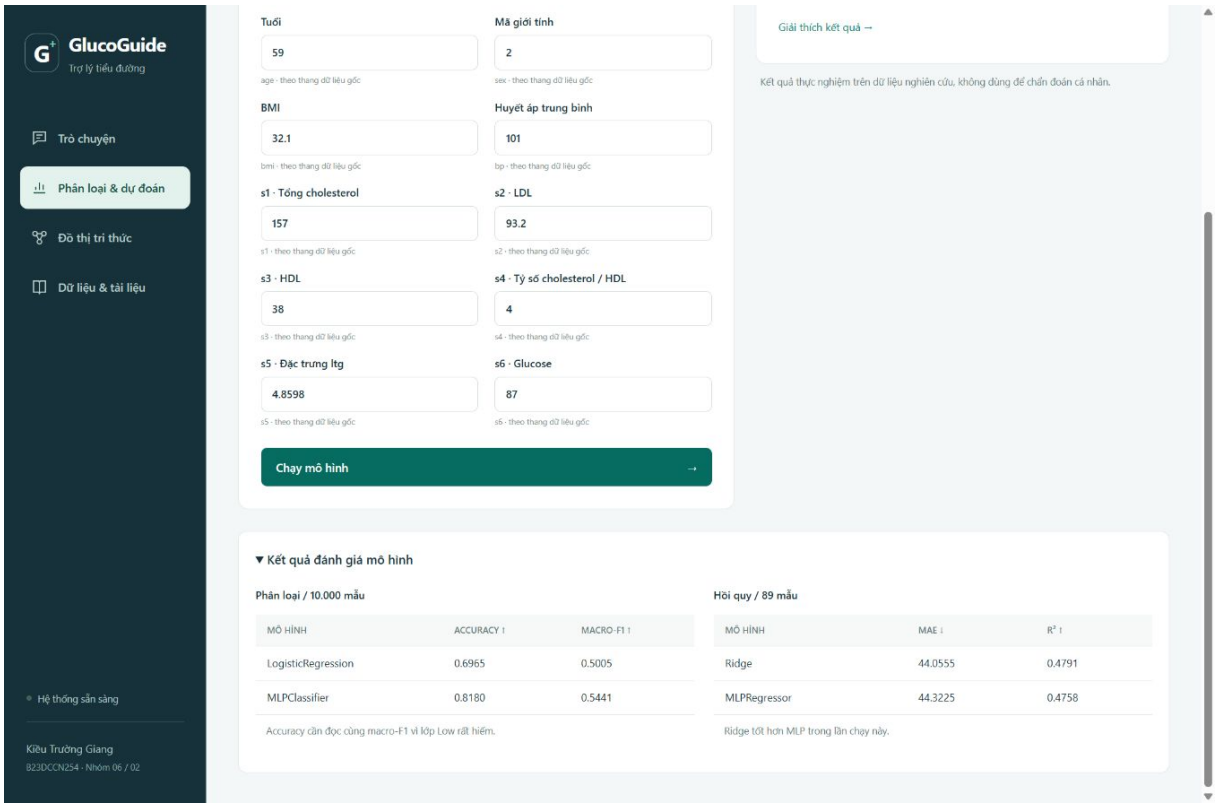
Hình 10.2: Màn hình phân loại: mẫu đầu vào và nhãn do checkpoint MLP trả về.

10.1.3. Thực nghiệm dữ liệu: hồi quy

Mở giao diện thực nghiệm

`http://localhost:8765/#models`

Chuyển sang "Dự đoán điểm tiến triển" để tạo biểu mẫu mười trường. Bấm điền mẫu lại sau khi đổi bài toán; không tái sử dụng tám giá trị của mô hình phân loại. Các tên s1 đến s6 giữ nguyên theo dữ liệu nghiên cứu. Đầu ra là một số thực, được làm tròn hai chữ số trên giao diện; đây là điểm tiến triển sau một năm của bộ diabetes. Bảng kết quả công bố cả Ridge và MLP để thể hiện rằng mô hình sâu không mặc nhiên tốt hơn đối chứng.



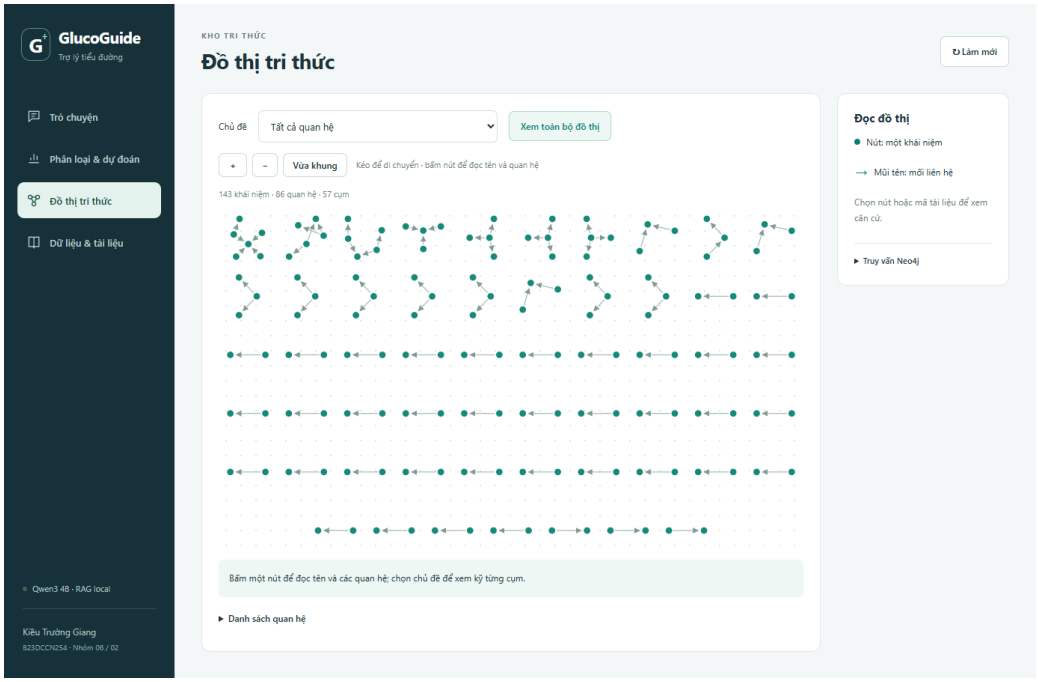
Hình 10.3: Màn hình hồi quy: biểu mẫu riêng và điểm tiến triển nghiên cứu.

10.1.4. Đồ thị tri thức và đối chiếu Neo4j

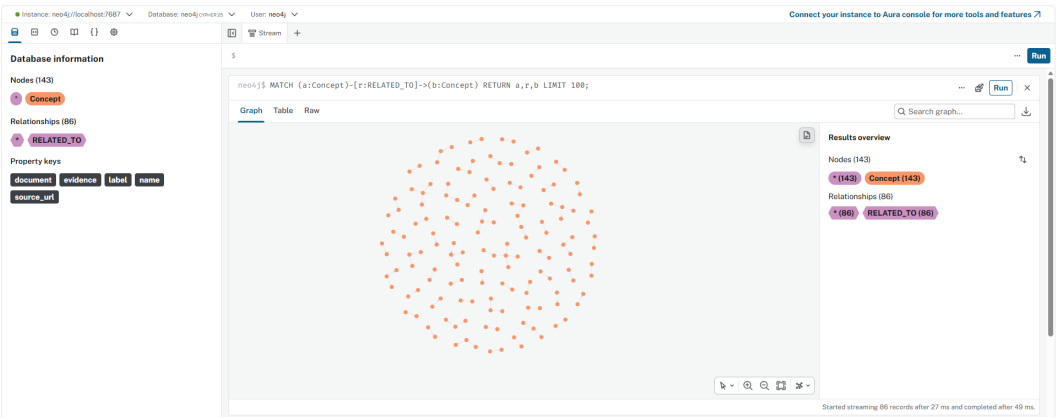
Mở giao diện thực nghiệm

`http://localhost:8765/#graph`

Khu vực đồ thị đọc quan hệ trực tiếp từ Neo4j qua GET /api/graph. Nút "Xem toàn bộ đồ thị" nằm cạnh bộ lọc Chủ đề; các nút phóng to, thu nhỏ và vừa khung hỗ trợ duyệt 143 khái niệm và 86 quan hệ. Người xem có thể kéo khung, bấm khái niệm để đọc liên kết hoặc chọn tài liệu để xem rõ tên và căn cứ. Giao diện web và Neo4j Browser dùng cùng dữ liệu nhưng có cách bố trí nút khác nhau.



Hình 10.4: Toàn bộ đồ thị trên web với nút xem nhanh và điều khiển phóng to.



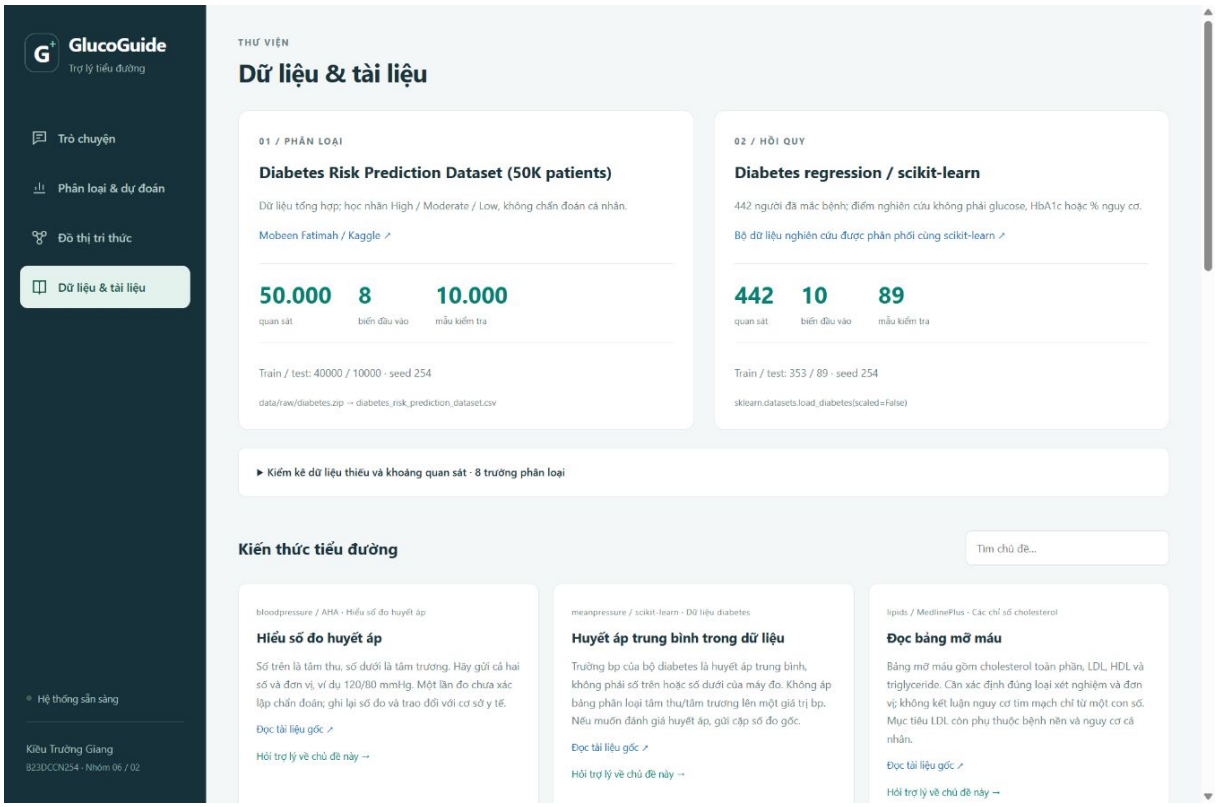
Hình 10.5: Neo4j Browser trả về cùng 143 khái niệm và 86 quan hệ.

10.1.5. Dữ liệu và nguồn

Mở giao diện thực nghiệm
`http://localhost:8765/#data`

Màn hình thứ tư công bố liên kết bộ dữ liệu, số dòng, số biến, số mẫu train/test và ý nghĩa nhãn. Bảng kiểm kê tám trường phân loại cho biết số giá trị thiếu cùng min, trung vị, max của bản chụp dữ liệu. Danh mục KB thống kê 19 nguồn được sử dụng trong 46 tài liệu. Mỗi tài liệu có mã để đối chiếu văn bản với quan hệ trên đồ thị.

Nội dung được xuất bởi scripts/audit_data.py thành web/data_catalog.json đồng thời với results/data_audit.json. Khi thay dữ liệu, cần chạy lại bước kiểm kê để đồng bộ báo cáo và web. Kho tri thức hiện là các đoạn văn đã biên soạn; hệ thống chưa có chức năng tải sách hoặc tự thu thập website trong phiên chat.



Hình 10.6: Màn hình công bố nguồn gốc và phạm vi hai bộ dữ liệu.

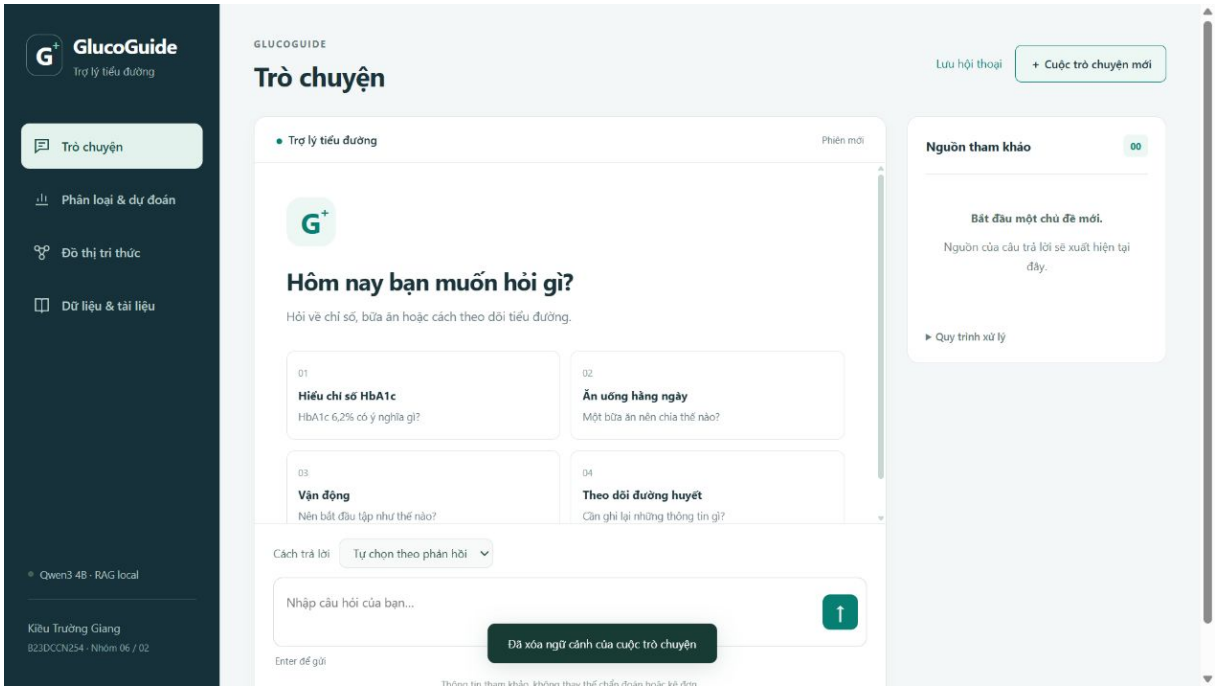
10.2. Thiết kế và vận hành chatbot tiểu đường

Chatbot kết hợp hội thoại có ngữ cảnh, tra cứu tài liệu và giải thích kết quả mô hình. Người dùng có thể hỏi tiếp về chủ đề đang trao đổi, đọc căn cứ ngay bên cạnh câu trả lời, hoặc chuyển kết quả thực nghiệm sang hội thoại để tìm hiểu ý nghĩa và giới hạn của đầu ra.

Giao diện sử dụng nền sáng, chữ không chân và màu xanh cho các thao tác chính. Thanh điều hướng bên trái phân chia bốn nhiệm vụ. Câu hỏi gợi ý nằm trong vùng hội thoại; nguồn đối chiếu ở cột bên phải, quy trình xử lý được thu gọn. Trên điện thoại, thanh điều hướng chuyển lên đầu trang và nội dung xếp theo một cột.

Truy cập trợ lý tiểu đường

`http://localhost:8765/#chat`



Hình 10.7: Giao diện trợ lý tiểu đường với thanh điều hướng, vùng hội thoại và tài liệu đối chiếu.

10.2.1. Ngữ cảnh, phạm vi và cơ chế trả lời

API tiếp nhận history gồm tối đa 64 thông điệp, mỗi thông điệp tối đa 3.000 ký tự, role chỉ là user hoặc assistant. Khi câu hiện tại là câu hỏi tiếp nối ngắn, lớp luật tìm chủ đề có thể nhận diện trong những câu hỏi người dùng trước đó. Ví dụ, sau câu "HbA1c là gì?", câu "Vậy 7,1% thì sao?" được hiểu là tiếp tục về HbA1c. Các con số cũ không tự điền vào câu mới; hệ thống không dựng hồ sơ lâm sàng đầy đủ từ lịch sử.

Lịch sử nằm trong bộ nhớ JavaScript của tab, không được ghi vào cơ sở dữ liệu. Nút cuộc

trò chuyện mới xóa lịch sử ở giao diện. Tải lại trang cũng bắt đầu phiên mới. Nút nguồn tham khảo mở trích đoạn và liên kết gốc ngay dưới câu trả lời. Nút lưu hội thoại tạo tệp văn bản cục bộ theo thao tác người dùng. Máy chủ xử lý lịch sử trong từng yêu cầu; đây chưa phải hệ thống quản lý bệnh án hoặc bộ nhớ dài hạn.

Các module conversation.py và dialogue.py phối hợp nhận diện chủ đề, xét câu hỏi tiếp nối và kiểm tra đơn vị. Bộ nhận diện ý định đã huấn luyện hỗ trợ một số câu giao tiếp; luật tiếp tục giữ quyền xử lý các nhánh có ràng buộc. Bộ truy xuất kết hợp điểm TF–IDF từ và ký tự để chọn tài liệu. Nhật ký ghi lại các yêu cầu qua MessageBus, kết quả nhận được và lỗi kết nối nếu có. Các vai trò này hoạt động trong cùng một tiến trình, chưa phải những dịch vụ tự chủ độc lập.

Bản local dùng Qwen3 4B để diễn đạt câu kiến thức từ tài liệu truy xuất. Các câu về thông số, thuốc hoặc tình huống cần hành động ưu tiên sử dụng phản hồi có kiểm soát bằng luật. Nếu bản sinh không đạt kiểm tra nhất quán, hệ thống thử sửa một lần rồi trở về nội dung KB khi cần. Cách tổ chức này giữ được phần hội thoại linh hoạt trong phạm vi nguồn đã có; chưa thay thế tư vấn chuyên môn trực tiếp.

10.2.2. Kiểm tra chỉ số trước khi giải thích

HbA1c phải có đơn vị phần trăm; nếu thiếu đơn vị, chatbot hỏi lại thay vì tự coi là phần trăm. Đường huyết nhận mg/dL hoặc mmol/L; quy đổi xấp xỉ theo hệ số 18 được công bố trong câu trả lời. Số đo sau ăn hoặc không rõ thời điểm không bị tự áp bảng lúc đói. Khi phát hiện nội dung thai kỳ hoặc trẻ em trong ngữ cảnh được xét, hệ thống không dùng bảng người lớn để diễn giải cá nhân.

Các ngưỡng và lưu ý được rà soát theo NIDDK; hướng dẫn hạ đường huyết tham chiếu CDC. Luật về dấu hiệu khẩn cấp ưu tiên lời nhắc tìm trợ giúp y tế. Câu hỏi liều thuốc được chuyển về giải thích phạm vi và thông tin chuẩn bị cho bác sĩ. Luật nhận diện bằng từ khóa vẫn có thể bỏ sót cách diễn đạt chưa kiểm thử; kết quả đạt kiểm thử không phải chứng nhận an toàn lâm sàng.

10.2.3. Nối mô hình với hội thoại và so sánh đầu vào

Sau khi suy luận, người dùng có thể bấm “Giải thích kết quả” để xem giải thích phạm vi và căn cứ. Thao tác giữ kết quả thực từ API, không dựng một dự đoán mới trong JavaScript. Người dùng cũng có thể đổi một trường và chạy lại để xem khác biệt so với lần chạy trước. Với mẫu hồi quy đã lưu, đổi BMI từ 32,1 thành 28 cho đầu ra từ 207,53 thành 179,64, chênh lệch khoảng -27,89 điểm trong lượt demo được kiểm tra. Đây là độ nhạy của mô hình trên đầu vào giả định; không suy ra hiệu quả điều trị hay mục tiêu giảm cân.

10.2.4. Tra cứu KB và sử dụng Neo4j

Mục Nguồn & tài liệu có tra cứu không dấu trên 46 đoạn kiến thức. Mỗi kết quả có nội dung, nguồn gốc và nút hỏi trợ lý. GET /api/knowledge trả các đoạn văn và thông tin nguồn; GET /api/status kiểm tra kết nối Neo4j và số lượng nút, cạnh.

Neo4j cục bộ lưu đồ thị của bản đang kiểm tra; AuraDB Free thuộc triển khai công khai trước đây. Người dùng có thể xem toàn bộ hoặc lọc một tài liệu ngay trên web, không cần nhớ câu lệnh Cypher. Đồ thị local có 143 khái niệm và 86 quan hệ, chia thành 57 thành phần liên thông. Toàn bộ 46 tài liệu đều có quan hệ, nhưng nhiều khái niệm vẫn thuộc các cụm nhỏ riêng biệt. Bố cục chỉ sắp xếp lại vị trí hiển thị, không tự thêm liên kết giữa các cụm.

Khi dịch vụ tắt, chạy start-neo4j.ps1 từ thư mục project. Để kiểm tra bằng Neo4j Browser, mở localhost:7474/browser và dùng tài khoản trong cấu hình cục bộ. Mật khẩu không được đưa vào báo cáo hoặc GitHub.

10.2.5. Kiểm thử bổ sung và thiết bị

Kiểm thử hội thoại gồm 37 điều kiện. Bộ kiểm tra an toàn gồm 29 điều kiện, chạy cùng 81 điều kiện hồi quy trong môi trường chặn kết nối ra ngoài. Kết quả lưu tại `results/safety_offline.json`.

Bộ kiểm tra giao diện đồ thị đối chiếu số nút và cạnh của đủ 46 bộ lọc tài liệu với dữ liệu Neo4j; kiểm tra nút xem toàn bộ, làm mới, phóng to, kéo, đọc khái niệm và mở nguồn. Nút điều khiển được kiểm tra ở chiều rộng 1.024 và 390 px, không bị đẩy khỏi màn hình hoặc gây tràn ngang. Kết quả nằm tại `results/graph_ui_checks.json`; phiên kiểm tra trình duyệt chỉ truy cập máy cục bộ.

Kiểm tra thủ công trên web bao gồm bấm nguồn sang đồ thị, chọn nút, độ phủ quan hệ của từng tài liệu, phân loại, hồi quy, thay đổi đầu vào, chuyển kết quả sang chat, tìm kiếm không dấu và xóa phiên. Bố cục được kiểm tra tại viewport desktop 1366 px và điện thoại 390 px; trang chat không bị tràn ngang ở kích thước điện thoại kiểm tra. Chưa kiểm thử toàn bộ trình duyệt hay đánh giá trải nghiệm với người dùng thật.



Hình 10.8: Bố cục điện thoại: điều hướng hai hàng và hội thoại một cột.

10.2.6. Hội thoại tiếp nối và đối chiếu nhiều chỉ số

Lớp hội thoại nhận tối đa 64 thông điệp trong phiên. Khi người dùng hỏi "đường huyết 7" rồi bổ sung "mmol/L, lúc đói", hệ thống ghép phần thông tin làm rõ với câu hỏi đang xét, công bố đơn vị và điều kiện áp dụng. Khi người dùng đưa một số đo mới, giá trị mới được xét riêng, không tự sao chép đơn vị từ số đo cũ. Với câu chứa cả HbA1c và đường huyết, mỗi chỉ số được đối chiếu theo loại xét nghiệm tương ứng, sau đó tổng hợp thành một phản hồi.

Nút nguồn dưới từng câu trả lời mở trực tiếp trích đoạn cùng liên kết đến tài liệu gốc. Nhờ đó người xem có thể kiểm tra căn cứ ngay cả trên màn hình hẹp. Bản công khai tổng hợp bằng luật và tài liệu. Nhánh local bổ sung Qwen3 4B với bước kiểm tra nhất quán; kết quả huấn luyện và giới hạn được trình bày trong chương riêng. Các câu nhiều ý, phủ định hoặc thay đổi chủ thể vẫn cần đánh giá thêm.

Hình 10.1 minh họa cách mở trích đoạn và liên kết gốc của một lượt hỏi.

Tài liệu tham khảo

[1] Kiều Trường Giang, Tiểu luận 02, nhóm 06–02, báo cáo học phần.

[2] Kiều Trường Giang, Tiểu luận 03, B23DCCN254, báo cáo và thực nghiệm học phần.

[3] Phạm Mai Linh, Tiểu luận 01, nhóm 06–03, báo cáo học phần.

[4] Nguyễn Tự Kiên, Tiểu luận final CT_07 và Project01 CT_07, báo cáo học phần.

[5] Bùi Văn Lâm, Tiểu luận final CT_03 và Project01 CT_03, mã 477, báo cáo học phần.

[6] E. J. Hu và cộng sự, LoRA: Low-Rank Adaptation of Large Language Models, 2021. https://arxiv.org/abs/2106.09685.

[7] Qwen Team, Qwen3 4B GGUF, model card. https://huggingface.co/Qwen/Qwen3-4B-GGUF.

[8] Qwen Team, Qwen3 1.7B, model card và hướng dẫn suy luận. https://huggingface.co/Qwen/Qwen3-1.7B.

[9] T. Kipf và M. Welling, Semi-Supervised Classification with Graph Convolutional Networks, ICLR 2017. https://arxiv.org/abs/1609.02907.

[10] W. Hamilton, R. Ying và J. Leskovec, Inductive Representation Learning on Large Graphs, 2017. https://arxiv.org/abs/1706.02216.

[11] P. Velickovic và cộng sự, Graph Attention Networks, ICLR 2018. https://arxiv.org/abs/1710.10903.

[12] M. Schlichtkrull và cộng sự, Modeling Relational Data with Graph Convolutional Networks, 2017. https://arxiv.org/abs/1703.06103.

[13] Scikit-learn, tài liệu load_diabetes. https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_diabetes.html.

[14] Mobeen Fatimah, Diabetes Risk Prediction Dataset 50K Patients. https://www.kaggle.com/datasets/mobeenfatimah/diabetes-risk-prediction-dataset-50k-patients.

[15] NIDDK, Diabetes Tests and Diagnosis. https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis.

[16] Danh mục nguồn tri thức của project được lưu tại data/knowledge.json; mỗi đoạn văn có mã tài liệu, mã nguồn và các bộ ba có căn cứ.

Phụ lục A

Mã nguồn tái lập

A.1. scripts/ingest_graph.py

```
from pathlib import Path
import json,os,sys
from neo4j import GraphDatabase
root=Path(__file__).resolve().parents[1]
sys.path.insert(0,str(root))
from app.settings import database
from app.retrieval import load_knowledge
env=os.environ
kb=load_knowledge()
assert all(d.get('triples') for d in kb['documents']), 'Every published topic must have grounded
↪  relationships'
rows=[dict(source=t[0],label=t[1],target=t[2],document=d['id'],evidence=d['text'],source_url=kb['sourc
↪  es'][d['source']]['url']) for d in kb['documents'] for t in
↪  d['triples']]
constraint='CREATE CONSTRAINT concept_name IF NOT EXISTS FOR (c:Concept) REQUIRE c.name IS UNIQUE'
ingest='UNWIND $rows AS row MERGE (a:Concept {name:row.source}) MERGE (b:Concept {name:row.target})
↪  MERGE (a)-[r:RELATED_TO {label:row.label,document:row.document}]->(b) SET r.evidence=row.evidence,
↪  r.source_url=row.source_url'
if '--https' in sys.argv:
 import httpx
 from urllib.parse import urlparse,quote
 endpoint='https://'+urlparse(env['NEO4J_URI']).hostname+'/db/'+quote(database(),safe='')+'/query/v2'
 with httpx.Client(auth=(env['NEO4J_USER'],env['NEO4J_PASSWORD']),timeout=30) as client:
  for statement,parameters in [(constraint,{}),(ingest,{'rows':rows}),('MATCH (n:Concept) RETURN
   ↪  count(n) AS nodes',{})]:
   response=client.post(endpoint,json={'statement':statement,'parameters':parameters})
   response.raise_for_status();result=response.json()
   if result.get('errors'):raise RuntimeError(result['errors'])
  print(result['data'])
else:
 with GraphDatabase.driver(env['NEO4J_URI'],auth=(env['NEO4J_USER'],env['NEO4J_PASSWORD']),connection_
  ↪  timeout=5,max_transaction_retry_time=5) as
  ↪  driver:
  driver.execute_query(constraint,database_=database())
  driver.execute_query(ingest,rows=rows,database_=database())
  print(driver.execute_query('MATCH (n:Concept) RETURN count(n) AS
   ↪  nodes',database_=database()).records)
(root/'results/graph_import.json').write_text(json.dumps({'documents':len(kb['documents']),'relations'
↪  :len(rows),'storage':'Neo4j'}))
```

A.2. scripts/train_models.py

```
"""Train reproducible MLP classifier and regression baseline on distinct datasets."""
from pathlib import Path
```

```
import json, zipfile, warnings
import numpy as np
import pandas as pd
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier, MLPRegressor
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.metrics import accuracy_score, f1_score, mean_absolute_error, r2_score
import joblib
ROOT=Path(__file__).resolve().parents[1]
source=ROOT/'data/raw/diabetes.zip'
with zipfile.ZipFile(source) as z:
 df=pd.read_csv(z.open(z.namelist()[0]))
features=['Age','BMI','HbA1c','Fasting_Blood_Sugar','Blood_Glucose','Blood_Pressure_Systolic','HDL','L
↪ DL']
X=df[features];y=df.Diabetes_Risk
tr,te=train_test_split(np.arange(len(df)),test_size=.2,stratify=y,random_state=254)
rows=[]
for name,est in
↪ [('LogisticRegression',LogisticRegression(max_iter=1000,class_weight='balanced')),('MLPClassifier'
↪ ,MLPClassifier(hidden_layer_sizes=(64,32,16),early_stopping=True,max_iter=150,random_state=254))]:
 model=make_pipeline(SimpleImputer(strategy='median'),StandardScaler(),est)
 model.fit(X.iloc[tr],y.iloc[tr]); pred=model.predict(X.iloc[te])
 rows.append(dict(task='classification',model=name,n_train=len(tr),n_test=len(te),accuracy=accuracy_sc
 ↪ ore(y.iloc[te],pred),macro_f1=f1_score(y.iloc[te],pred,average='macro')))
 joblib.dump(model,ROOT/'models'/f'{name}.joblib')
 if name.startswith('MLP'): pd.DataFrame({'row':te,'actual':y.iloc[te].values,'predicted':pred}).to_cs
 ↪ v(ROOT/'results/classification_predictions.csv',index=False)
d=load_diabetes(scaled=False)
tr,te=train_test_split(np.arange(len(d.target)),test_size=.2,random_state=254)
for name,est in [('Ridge',Ridge(alpha=10)),('MLPRegressor',MLPRegressor(hidden_layer_sizes=(64,32,16),
↪ max_iter=1000,early_stopping=True,random_state=254))]:
 model=make_pipeline(StandardScaler(),est);model.fit(d.data[tr],d.target[tr]);pred=model.predict(d.dat
 ↪ a[te])
 rows.append(dict(task='regression',model=name,n_train=len(tr),n_test=len(te),mae=mean_absolute_error(
 ↪ d.target[te],pred),r2=r2_score(d.target[te],pred)))
 joblib.dump(model,ROOT/'models'/f'{name}.joblib')
 if name.startswith('MLP'): pd.DataFrame({'row':te,'actual':d.target[te],'predicted':pred}).to_csv(ROO
 ↪ T/'results/regression_predictions.csv',index=False)
(ROOT/'results/model_metrics.json').write_text(json.dumps(rows,indent=2))
(ROOT/'models/schema.json').write_text(json.dumps({'classification':features,'regression':list(d.featu
↪ re_names),'seed':254,'classification_scope':'synthetic dataset
↪ labels','regression_scope':'research progression score, not glucose or personal disease
↪ probability'},indent=2))
print(json.dumps(rows,indent=2))
```

A.3. scripts/train_policy.py

```
from pathlib import Path
import json,numpy as np
root=Path(__file__).resolve().parents[1]
rng=np.random.default_rng(254);q=np.zeros((4,3));history=[]
```

```
# Explicit simulated workflow: missing evidence -> clarify; evidence -> cite;
# prediction plus evidence -> explain model and cite. Not a treatment policy.
for ep in range(3000):
 s=int(rng.integers(4));a=int(rng.integers(3)) if rng.random()<.2 else int(q[s].argmax())
 target=0 if s%2==0 else (2 if s==3 else 1)
 reward=1. if a==target else -1.
 q[s,a]+=.15*(reward-q[s,a]);history.append(reward)
payload={'seed':254,'episodes':3000,'q':q.tolist(),'actions':['Hỏi làm rõ','Trả lời có nguồn','Giải
↪  thích mô hình và dẫn nguồn'],'environment':'One-step simulated workflow, not clinical
↪  outcomes','last_300_reward':float(np.mean(history[-300:]))}
(root/'models/workflow_policy.json').write_text(json.dumps(payload,ensure_ascii=False,indent=2),encodi
↪  ng='utf-8')
print(payload['last_300_reward'])
```

A.4. app/agents.py

```
from dataclasses import dataclass,field
from typing import Callable,Any
@dataclass
class Message:
 sender:str
 receiver:str
 type:str
 content:Any
@dataclass
class Agent:
 name:str
 handler:Callable
 memory:dict=field(default_factory=dict)
 def receive(self,message,bus):
  self.memory['calls']=self.memory.get('calls',0)+1
  self.memory['last_request']=message.type
  return self.handler(message.content,bus)
class MessageBus:
 def __init__(self):self.agents={};self.trace=[]
 def register(self,agent):self.agents[agent.name]=agent
 def request(self,sender,receiver,content,kind='REQUEST'):
  self.trace.append(vars(Message(sender,receiver,kind, str(content) if isinstance(content,dict) and
  ↪  'x' in content else content)))
  try:
   result=self.agents[receiver].receive(Message(sender,receiver,kind,content),self)
   self.trace.append(vars(Message(receiver,sender,'INFORM',result.tolist() if hasattr(result,'tolist')
   ↪  else result)))
   return result
  except Exception as exc:
   self.trace.append(vars(Message(receiver,sender,'FAILURE',type(exc).__name__)))
   raise
```

A.5. app/conversation.py

```
"""Conversation routing with bounded user-turn context and cited, deterministic responses."""
import re, unicodedata
from app.intent import predict_intent
```

```
def norm(s):
 return ''.join(c for c in unicodedata.normalize('NFD',s.lower().replace('đ','d')) if
  ↪ unicodedata.category(c)!='Mn')

TOPICS=[('meanpressure',['huyet ap trung binh','bien bp','truong
↪ bp'],['meanpressure']),('bloodpressure',['huyet ap','tam thu','tam
↪ truong','mmhg'],['bloodpressure']),('a1c',['hba1c','a1c'],['a1c']),('meal',['an gi','an uong','bua
↪ an','thuc don','com','tinh bot','dinh duong'],['plate','carb']),('fruit',['trai cay','hoa
↪ qua','nuoc ep'],['fruit','carb']),('activity',['van dong','the duc','di bo','tap luyen'],['activit
↪ y']),('medicine',['thuoc','insulin','metformin','lieu'],['medicine']),('symptoms',['trieu
↪ chung','khat nhieu','tieu nhieu','sut can'],['symptoms','testing']),('low',['ha duong','duong
↪ huyet thap','run tay','va mo hoi'],['low','urgent']),('testing',['duong huyet','glucose','xet
↪ nghiem','do tieu duong','kiem tra tieu duong'],['testing']),('monitor',['theo doi','ghi
↪ chep','nhat ky','do luc nao'],['monitor']),('model',['diem tien trien','hoi
↪ quy'],['model']),('diabetes',['tieu duong','dai thao
↪ duong'],['overview','testing']),('classification',['nhan high','nhan low','nhan moderate','phan
↪ loai','nhan tong hop'],['classification_scope'])]
TOPICS=[('sick_day',['bi om','ngay om','bi sot','sot','non khi bi tieu
↪ duong'],['sick_day']),('foot_care',['ban chan','te chan','vet loet o chan','kiem tra
↪ chan'],['foot_care']),('kidney_checks',['uacr','egfr','albumin nieu','chuc nang than','benh
↪ than','xet nghiem than'],['kidney_checks'])]+[(key,terms,ids+['activity_plan'] if key=='activity'
↪ else ids) for key,terms,ids in TOPICS]
FOLLOW={
 'bloodpressure':['Đo huyết áp tại nhà thế nào?','Tôi nên ghi gì khi đi khám?'],
 'a1c':['Ngưỡng HbA1c được hiểu thế nào?','Tôi nên chuẩn bị gì khi đi khám?'],
 'meal':['Có cần bỏ cơm hoàn toàn không?','Người tiểu đường ăn trái cây được không?'],
 'fruit':['Gợi ý cách chia một bữa ăn','Tôi nên theo dõi đường huyết thế nào?'],
 'testing':['Đo lúc đói khác sau ăn thế nào?','HbA1c là gì?'],
 'medicine':['Tôi nên ghi gì để trao đổi với bác sĩ?','Dấu hiệu hạ đường huyết là gì?'],
 'low':['Theo dõi để tránh hạ đường huyết thế nào?'],
 'activity':['Tôi nên theo dõi đường huyết thế nào?'],
 'monitor':['Gợi ý cách chia một bữa ăn','HbA1c là gì?'],
 'symptoms':['Tôi nên chuẩn bị gì khi đi khám?'],
 'classification':['Nhãn High có phải chẩn đoán không?'],
 'diabetes':['HbA1c là gì?','Tôi nên theo dõi đường huyết thế nào?'],
 'model':['Điểm tiến triển có phải đường huyết không?']}
FOLLOW.update({'sick_day':['Tôi nên ghi gì khi bị ốm?'],'foot_care':['Tôi cần kiểm tra bàn chân ra
↪ sao?'],'kidney_checks':['UACR và eGFR dùng để làm gì?']})

def topic(q):
 for key,terms,ids in TOPICS:
  if any(re.search(r'(?<!\w)'+re.escape(t)+r'(?!\w)',norm(q)) for t in terms):return key,ids
 if re.search(r"\d|so duoi|so tren|thi sao|chi tiet|giai thich them",norm(q)):return None,[]
 learned=predict_intent(norm(q))
 if learned:
  return next((key,ids) for key,terms,ids in TOPICS if key==learned)
 return None,[]

def _route(question,history):
 q=norm(question); key,ids=topic(question);used=False; prior=''
 if any(t in q for t in ['lieu thuoc','lieu insulin','bao nhieu don vi','tang lieu','giam
  ↪ lieu']):key,ids='medicine',['medicine']
 # Only the most recent user question can supply a topic; never reuse old numeric values.
 previous=[h['content'] for h in history if h['role']=='user']
 if previous:
  prior=next((text for text in reversed(previous) if topic(text)[0]),previous[-1])
 if not key and prior and (len(q)<90 and any(t in q for t in ['vay','the con','chi so do','no la','co
  ↪ cao','bao nhieu','nghia la','nen lam','6.','7.','%','so duoi','so tren','con ','thi sao','co
  ↪ on'])):
```

```
 key,ids=topic(prior);used=bool(key)
query=(key+' '+question) if used else question
if used:query=prior+' '+question
result={'topic':key or 'knowledge','ids':ids,'query':query,'context_used':used,'intro':'','override':
↪  None,'tone':'normal','suggestions':FOLLOW.get(key,['HbA1c là gì?','Gợi ý cách chia một bữa
↪  ăn','Tôi nên theo dõi đường huyết thế nào?'])}
# Active emergency wording is separate from an educational question about symptoms.
urgent=any(t in q for t in ['bat tinh','co giat','khong nuot duoc','kho tho','lu lan'])
educational=any(t in q for t in ['la gi','dau hieu','neu ','khi nao','khong bi','khong co','khong kho
↪  tho'])
if urgent and not educational:
 result.update(topic='urgent',ids=['urgent'],tone='urgent',intro='Tình huống bạn mô tả có thể cần trợ
 ↪  giúp y tế khẩn cấp. Đừng chờ câu trả lời tiếp theo trên chat.',suggestions=[]);return result
if re.fullmatch(r'(xin chao|chao|hello|hi|cam on|thank you)[!. ]*',q):
 result.update(override='Chào bạn. Tôi có thể cùng bạn tìm hiểu tiểu đường: giải thích HbA1c, cách ăn
 ↪  uống, theo dõi đường huyết và đọc ý nghĩa kết quả nghiên cứu. Bạn đang muốn hỏi về chỉ số hay
 ↪  sinh hoạt hằng ngày?',ids=[]);return result
if 'chuan bi' in q or 'ghi gi' in q:
 result.update(topic='monitor',ids=['monitor','medicine'],intro='Để buổi khám hữu ích hơn, hãy chuẩn
 ↪  bị phiếu xét nghiệm và ghi lại các thông tin sau.');return result
if key=='testing' and any(t in q for t in ['do tieu duong','kiem tra tieu duong']):
 result.update(ids=['testing','a1c'],override='Để kiểm tra tiểu đường, bác sĩ thường dùng xét nghiệm
 ↪  đường huyết lúc đói hoặc HbA1c. Bạn cần ghi tên xét nghiệm, kết quả, đơn vị, thời điểm đo và có
 ↪  nhịn ăn hay không. Máy đo tại nhà dùng để theo dõi, không tự xác nhận bệnh. Bạn đang có phiếu
 ↪  HbA1c hay một số đo đường huyết?');return result
if key=='medicine':
 result['intro']='Tôi có thể giải thích nguyên tắc dùng thuốc, nhưng không chọn thuốc hoặc tính liều
 ↪  cho bạn.'
# Avoid assigning adult screening ranges during pregnancy or in children.
special=any(t in q+' '+norm(prior) for t in ['mang thai','thai ky','tre em','be nha','con toi'])
if special and key in ['a1c','testing','low']:
 result.update(intro='Trẻ em và thai kỳ cần cách đánh giá riêng. Tôi không áp bảng ngưỡng người lớn
 ↪  cho trường hợp này; hãy trao đổi với cơ sở đang theo
 ↪  dõi.',ids=['demographics','monitor']);return result
if key=='bloodpressure':
 result.update(ids=['bloodpressure'])
 if special:
  result['override']='Với trẻ em hoặc thai kỳ, cách đánh giá huyết áp cần theo bối cảnh riêng. Bạn
  ↪  cho biết tuổi hoặc tuần thai, cả hai số đo và triệu chứng hiện tại; liên hệ cơ sở đang theo dõi
  ↪  để được hướng dẫn.';return result
 pair=re.search(r'(\d{2,3})\s*[/\-]\s*(\d{2,3})',q)
 # A short follow-up can explicitly provide the missing lower number.
 if not pair and used and re.search(r'so duoi|tam truong',q):
  upper=re.search(r'huyet ap\s*(\d{2,3})',norm(prior))
  lower=re.search(r'(?:so duoi|tam truong)\s*(?:la\s*)?(\d{2,3})',q)
  if upper and lower:pair=(upper[1],lower[1])
 if pair:
  top,bottom=int(pair[1] if not isinstance(pair,tuple) else pair[0]),int(pair[2] if not
  ↪  isinstance(pair,tuple) else pair[1])
  if not (40<=top<=300 and 20<=bottom<=200 and top>bottom):
   result['override']='Bạn kiểm tra lại hai số và thứ tự trên/dưới trên máy đo nhé; cặp số này chưa
   ↪  hợp lý để diễn giải.';return result
  if top>=180 or bottom>=120:
   result.update(tone='urgent',override=f'Số đo {top}/{bottom} mmHg rất cao. Nếu có đau ngực, khó
   ↪  thở, yếu liệt, nhìn mờ hoặc khó nói, hãy gọi cấp cứu địa phương ngay. Nếu không có các triệu
   ↪  chứng đó, nghỉ yên, đo lại sau ít nhất một phút và liên hệ nhân viên y tế ngay nếu số đo vẫn
   ↪  rất cao. Không tự tăng liều thuốc.',suggestions=[]);return result
  band='cao theo bảng tham chiếu AHA cho người lớn' if top>=130 or bottom>=80 else ('tăng nhẹ ở số
  ↪  trên' if top>=120 else 'dưới ngưỡng tăng huyết áp theo bảng AHA')
```

```
  result['override']=f'Nếu đây là số đo huyết áp ở người lớn, {top}/{bottom} mmHg là {band}. Một lần
   ↪ đo chưa xác lập chẩn đoán. Hãy nghỉ yên 5 phút, đo lại hai lần cách nhau một phút, ghi kết quả
   ↪ và trao đổi với nhân viên y tế nếu số đo cao lặp lại. Bạn đo khi nghỉ hay vừa vận động, và hiện
   ↪ có triệu chứng gì không?';return result
 number=re.search(r'(?<!\w)(\d{2,3})(?!\w)',q)
 if number:
  v=int(number[1]);result['override']=f'Bạn nói huyết áp {v}: đây là số trên (tâm thu) hay số dưới
   ↪ (tâm trương)? Hãy gửi cả cặp số và đơn vị, ví dụ 150/90 mmHg. Nếu {v} là số trên thì '+('đã cao
   ↪ theo bảng tham chiếu AHA cho người lớn; nên đo lại khi nghỉ và trao đổi với nhân viên y tế nếu
   ↪ lặp lại.' if v>=130 else 'cần xem thêm số dưới và điều kiện đo để nhận xét.')+' Bạn hiện có đau
   ↪ ngực, khó thở hoặc triệu chứng bất thường không?';return result
 if any(t in q for t in ['do the nao','cach do','tai nha']):
  result['override']='Để đo huyết áp tại nhà: nghỉ yên 5 phút, ngồi tựa lưng, hai chân đặt trên sàn,
   ↪ tay đỡ ngang tim và không nói chuyện khi đo. Tránh hút thuốc, caffeine và vận động trong 30
   ↪ phút trước đo. Dùng vòng bít phù hợp, đo hai lần cách nhau một phút và ghi cả số trên/số dưới
   ↪ cùng thời điểm.';return result
 result['override']='Huyết áp có hai số: số trên là tâm thu, số dưới là tâm trương, thường ghi bằng
  ↪ mmHg. Bạn gửi cả cặp số (ví dụ 120/80), thời điểm đo và triệu chứng hiện tại để tôi giải thích
  ↪ đúng bối cảnh.';return result
if key=='a1c':
 match=re.search(r'(?:hba1c|a1c)\s*(?:cua toi|la|=|:)?\s*(\d+(?:[.,]\d+)?)',q) or
  ↪ re.search(r'(\d+(?:[.,]\d+)?)\s*%',q)
 if match:
  value=float(match[1].replace(',','.'))
  if '%' not in q:
   result.update(override='Bạn xác nhận HbA1c này có đơn vị % không? Nếu phiếu ghi mmol/mol, cần diễn
    ↪ giải theo đơn vị đó; tôi chưa áp ngưỡng khi thiếu đơn vị.',ids=['a1c']);return result
  if not 2<=value<=25:
   result.update(override='Giá trị HbA1c này nằm ngoài miền mà bản demo xử lý. Bạn kiểm tra lại tên
    ↪ xét nghiệm, con số và đơn vị trên phiếu nhé.',ids=['a1c']);return result
  band='dưới ngưỡng tiền đái tháo đường' if value<5.7 else ('trong khoảng sàng lọc tiền đái tháo
   ↪ đường' if value<6.5 else 'đạt ngưỡng xét nghiệm dùng để chẩn đoán đái tháo đường')
  result['intro']=f'HbA1c {value:g}% {band} theo bảng tham chiếu NIDDK cho người không mang thai. Đây
   ↪ chưa phải kết luận chẩn đoán của riêng bạn. Nếu bạn đã được chẩn đoán, mục tiêu điều trị cần
   ↪ trao đổi riêng với bác sĩ.'
if key in ['testing','low']:
 match=re.search(r'(\d+(?:[.,]\d+)?)\s*(mg\s*/?\s*dl|mmol\s*/?\s*l)',q)
 if match:
  value=float(match[1].replace(',','.'));mg=value*18 if 'mmol' in match[2] else value
  if mg<=0 or mg>1000:
   result.update(override='Bạn kiểm tra lại giá trị và đơn vị đường huyết trên phiếu. Con số này nằm
    ↪ ngoài miền xử lý của bản demo.',ids=['testing']);return result
  if mg<70:
   result.update(topic='low',ids=['low','urgent'],tone='urgent',intro=f'Số đo bạn nhập tương đương
    ↪ khoảng {mg:g} mg/dL, dưới mức 70 mg/dL mà CDC coi là thấp. Nếu đây là số đo hiện tại, cần xử
    ↪ trí ngay; nếu khó nuốt, lú lẫn hoặc bất tỉnh, nhờ người bên cạnh gọi cấp
    ↪ cứu.',suggestions=FOLLOW['low']);return result
  fasting=bool(re.search(r'\b(luc doi|nhin doi)\b',q)) and not any(t in q for t in ['khong
   ↪ doi','khong phai luc doi'])
  if fasting:
   band='dưới 100 mg/dL' if mg<100 else ('trong khoảng 100–125 mg/dL' if mg<126 else 'từ 126 mg/dL
    ↪ trở lên')
   result['intro']=f'Bạn ghi số đo lúc đói tương đương khoảng {mg:g} mg/dL, {band}. Bảng sàng lọc chỉ
    ↪ áp dụng cho xét nghiệm huyết tương sau nhịn ăn ít nhất 8 giờ. Máy đo tại nhà không tự xác lập
    ↪ chẩn đoán; hãy xác nhận loại xét nghiệm với cơ sở y tế.'
  else:result['intro']='Bạn đo lúc đói, sau ăn hay ngẫu nhiên, và đo tại nhà hay xét nghiệm ở cơ sở y
   ↪ tế? Tôi chưa áp ngưỡng lúc đói cho số đo thiếu bối cảnh.'
 elif re.search(r'\d',q):result['intro']='Bạn bổ sung đơn vị mg/dL hoặc mmol/L và thời điểm đo nhé.
  ↪ Một con số chưa đủ để diễn giải đường huyết.'
```

```
 return result


def contextual_route(question, history):
 """Resolve explicitly supplied follow-up fields, without silently inventing values."""
 q=norm(question)
 if re.fullmatch(r'(ban la ai|ban ten gi|ban lam gi|ban co the lam gi|gioi thieu( di| ban than)?|tu
  ↪  gioi thieu( di)?|noi ve ban( di)?)\s*[?.!]*',q.strip()):
  return {'topic':'social','ids':[],'query':question,'context_used':False,'intro':'','override':'Mình
   ↪  là GlucoGuide, trợ lý tìm hiểu về tiểu đường. Mình có thể giải thích chỉ số xét nghiệm, gợi ý
   ↪  cách theo dõi và tìm thông tin có nguồn trong kho tài liệu. Bạn muốn bắt đầu từ câu hỏi
   ↪  nào?','tone':'normal','suggestions':['Tiểu đường là gì?','HbA1c là gì?']}
 if re.fullmatch(r'(toi |mình )?(ghet|chan) ban[.!? ]*',q) or q.strip(' .!?') in ['do ngoc','ban tra
  ↪  loi sai','ban noi sai']:
  return {'topic':'social','ids':[],'query':question,'context_used':False,'intro':'','override':'Mình
   ↪  xin lỗi vì câu trả lời vừa rồi không đúng ý bạn. Bạn nói giúp mình chỗ nào sai hoặc muốn hỏi lại
   ↪  phần nào; mình sẽ trả lời sát câu đó hơn.','tone':'normal','suggestions':['Giải thích lại câu
   ↪  vừa rồi','Tôi cần đo những chỉ số nào?']}
 if q.strip(' .!?') in ['toi yeu ban','yeu ban','cam on ban','ban de thuong qua']:
  return {'topic':'social','ids':[],'query':question,'context_used':False,'intro':'','override':'Cảm
   ↪  ơn bạn đã quý mến. Tôi luôn sẵn sàng giúp bạn tìm hiểu tiểu đường và các chỉ số. Bạn muốn trao
   ↪  đổi điều gì tiếp theo?','tone':'normal','suggestions':['Tiểu đường là gì?','HbA1c là gì?']}
 users=[h['content'] for h in history if h['role']=='user']
 # Retain only the active topic; a new explicit topic starts a new context.
 anchor=None
 for i in range(len(users)-1,-1,-1):
  if topic(users[i])[0]:anchor=i;break
 current=topic(question)[0]
 # A short clarification refers to the last meaningful user question, not to random retrieval hits.
 if anchor is not None and re.fullmatch(r'(la sao|y la sao|y la gi|giai thich lai|toi khong hieu)[?.!
  ↪  ]*',q):
  old_topic=topic(users[anchor])[0]
  if old_topic=='testing':
   answer='Để kiểm tra tiểu đường, hai xét nghiệm thường gặp là đường huyết lúc đói (ghi kết quả cùng
    ↪  đơn vị mg/dL hoặc mmol/L) và HbA1c (thường ghi bằng %). Máy đo tại nhà giúp theo dõi nhưng
    ↪  không tự xác nhận bệnh. Bạn đang có kết quả nào? Gửi tên xét nghiệm, số đo, đơn vị và lúc đo;
    ↪  mình sẽ giải thích đúng chỉ số đó.'
   ids=['testing','a1c']
  elif old_topic=='meal':
   answer='Với bữa ăn, bạn có thể hình dung nửa đĩa là rau, một phần tư là đạm như cá hoặc đậu phụ,
    ↪  phần còn lại là cơm hay tinh bột. Đây là gợi ý chia đĩa, lượng ăn thực tế cần theo nhu cầu và
    ↪  thuốc của bạn. Bạn muốn mình giải thích phần nào?'
   ids=['plate','carb']
  elif old_topic=='a1c':
   answer='HbA1c cho biết mức đường huyết trung bình khoảng ba tháng. Để hiểu con số trên phiếu, bạn
    ↪  gửi thêm giá trị và đơn vị; kết quả đơn lẻ chưa đủ xác nhận chẩn đoán.'
   ids=['a1c']
  else:
   answer='Bạn muốn mình giải thích lại phần nào trong câu trả lời trước? Bạn có thể nhắc tên chỉ số
    ↪  hoặc chép đúng câu chưa rõ.'
   ids=[]
  return {'topic':old_topic or 'knowledge','ids':ids,'query':users[anchor],'context_used':True,'intro'
   ↪  :'','override':answer,'tone':'normal','suggestions':FOLLOW.get(old_topic,[])}
 if anchor is not None:
  old_topic=topic(users[anchor])[0]
  short=bool(re.fullmatch(r'[\d.,% /a-z]+',q)) and len(q)<45
  clarify=any(t in q for t in ['mmol','mg/dl','phan tram','luc doi','sau an','so duoi','so tren','noi
   ↪  ro','chi tiet','giai thich them','toi nen lam gi'])
  if not current and (short or clarify):
```

```
   recent=' '.join(users[anchor:])
   # Unit-only clarification attaches to the last measurement; a new number replaces it.
   if old_topic in ['a1c','testing'] and not re.search(r'\d',q) and any(t in q for t in
    ↪  ['mmol','mg/dl','%','phan tram','luc doi','sau an']):
    enriched=recent+' '+question.replace('phần trăm','%')
    result=_route(enriched,history);result['context_used']=True;return result
   if old_topic in ['a1c','testing'] and re.search(r'\d',q):
    result=_route(('HbA1c ' if old_topic=='a1c' else 'đường huyết
     ↪  ')+question,history);result['context_used']=True;return result
   if any(t in q for t in ['noi ro','chi tiet','giai thich them','toi nen lam gi']):
    result=_route(users[anchor],history);result['context_used']=True
    extra={
     'a1c':'Bước tiếp theo: kiểm tra đúng tên xét nghiệm và đơn vị trên phiếu, mang các kết quả trước
      ↪  đây khi đi khám, và trao đổi việc xét nghiệm xác nhận nếu kết quả cao. HbA1c phản ánh nhiều
      ↪  tuần, nên không đánh giá thay đổi chỉ sau một bữa ăn. Bạn đã được chẩn đoán tiểu đường trước
      ↪  đây hay đang kiểm tra lần đầu?',
     'testing':'Để đọc kết quả sát hơn, cần biết đơn vị, lúc đói hay sau ăn, thời gian nhịn ăn và đây
      ↪  là máy đo tại nhà hay xét nghiệm tại cơ sở y tế. Bạn đã có thông tin nào trong số này?',
     'bloodpressure':'Bạn nên ghi cả hai số đo, giờ đo, có vừa vận động hay không, thuốc đang dùng và
      ↪  triệu chứng. Nghỉ yên trước khi đo lại; kết quả cao lặp lại cần trao đổi với nhân viên y tế.
      ↪  Bạn đã đo lại lần thứ hai chưa?',
     'meal':'Bạn có thể bắt đầu từ một bữa quen thuộc: chia rau, đạm và phần tinh bột theo gợi ý trong
      ↪  nguồn. Không tự bỏ hết tinh bột, đặc biệt khi dùng thuốc dễ gây hạ đường huyết. Bạn thường ăn
      ↪  gì ở bữa chính và muốn điều chỉnh bữa nào?'}
    if old_topic in extra:
     result['override']=(result['override'] or result['intro'] or '')+'\n\n'+extra[old_topic]
    return result
 return _route(question,history)


def route(question, history):
 # Parse independent measurements separately, so one topic cannot hide the other.
 q=norm(question)
 if 'hba1c' in q and any(t in q for t in ['duong huyet','glucose']):
  a=re.search(r'(?:hba1c|a1c)\s*(?:la|:|=)?\s*\d+(?:[.,]\d+)?\s*%?',q)
  g=re.search(r'(?:duong huyet|glucose)\s*(?:luc doi|sau
   ↪  an)?\s*(?:la|:|=)?\s*\d+(?:[.,]\d+)?\s*(?:mg/dl|mmol/l)?',q)
  if a and g:
   plans=[contextual_route(x.group(),history) for x in [a,g]]
   # Special populations and urgent flags must still govern the whole question.
   guard=_route(question,history)
   if guard['tone']=='urgent' or any(t in q for t in ['mang thai','thai ky','tre em','con
    ↪  toi']):return guard
   result=plans[0].copy();result['ids']=list(dict.fromkeys(i for p in plans for i in p['ids']))
   result['override']='Đối chiếu từng chỉ số bạn cung cấp:\n\n'+'\n\n'.join((p['override'] or
    ↪  p['intro']) for p in plans)+'\n\nHai phép đo phản ánh những khoảng thời gian khác nhau. Hãy
    ↪  mang phiếu xét nghiệm để bác sĩ đối chiếu và xác nhận; không quy đổi điểm mô hình nghiên cứu
    ↪  thành chẩn đoán.'
   if any(p['tone']=='urgent' for p in plans):result.update(tone='urgent',suggestions=[])
   return result
 return contextual_route(question,history)
```

A.6. app/main.py

```
import json, os, math, re, unicodedata
from pathlib import Path
import numpy as np
```

```
import pandas as pd
import joblib
from fastapi import FastAPI, HTTPException
from fastapi.responses import FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field
from neo4j import GraphDatabase
from app.agents import Agent, MessageBus
from app.conversation import route
from app.dialogue import plan_turn
from app.answer_display import clean_answer
from app.retrieval import load_knowledge, Retriever
from app.graph_knowledge import GraphKnowledgeAgent
from app import local_llm, feedback
from app import safety
from app.model_explanation import explain
from app.settings import database, browser_url
from typing import Literal
ROOT=Path(__file__).resolve().parents[1]
for line in (ROOT/'.env').read_text().splitlines() if (ROOT/'.env').exists() else []:
 if '=' in line:
  k,v=line.split('=',1);os.environ.setdefault(k,v)
kb=load_knowledge()
docs=kb['documents']
retriever=Retriever(docs)
schema=json.loads((ROOT/'models/schema.json').read_text())
models={k:joblib.load(ROOT/'models'/f'{v}.joblib') for k,v in
 ↪  [('classification','MLPClassifier'),('regression','MLPRegressor')]}
app=FastAPI(title='GlucoGuide • Giang KT')
app.mount('/assets',StaticFiles(directory=ROOT/'web'),name='assets')
class Turn(BaseModel):
 role:Literal["user","assistant"]
 content:str=Field(max_length=3000)

class Request(BaseModel):
 question:str=Field(default='',max_length=3000)
 task:str='knowledge'
 values:dict=Field(default_factory=dict)
 history:list[Turn]=Field(default_factory=list,max_length=64)
 preferred_format:Literal['auto','brief','steps','explain']='auto'
def graph_driver():
 uri=os.environ.get('NEO4J_URI','bolt://127.0.0.1:7687')
 if os.environ.get('GLUCOGUIDE_LOCAL_ONLY')=='1':
  from urllib.parse import urlparse
  if urlparse(uri).hostname not in {'localhost','127.0.0.1','::1'}:raise RuntimeError('Local mode
   ↪  requires a local graph')
 return GraphDatabase.driver(uri,auth=(os.environ.get('NEO4J_USER','neo4j'),os.environ.get('NEO4J_PASS
  ↪  WORD','')),connection_timeout=3,connection_acquisition_timeout=4,max_transaction_retry_time=0)
@app.get('/')
def home():return FileResponse(ROOT/'web/index.html')
@app.get('/api/metrics')
def metrics():return json.loads((ROOT/'results/model_metrics.json').read_text())
@app.get('/api/schema')
def fields():return schema
@app.get('/api/examples')
def examples():return json.loads((ROOT/'data/examples.json').read_text(encoding='utf-8'))
@app.get('/api/graph')
def graph():
 try:
```

```
  with graph_driver() as driver:
   rows=driver.execute_query('MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept) RETURN a.name AS
    ↪  source,b.name AS target,r.label AS label,r.document AS document LIMIT 500',
    ↪  database_=database()).records
   edges=[dict(r) for r in rows]
   return {'storage':'Neo4j','nodes':sorted(set(e[k] for e in edges for k in
    ↪  ['source','target'])),'edges':edges}
 except Exception:raise HTTPException(503,'Chưa kết nối được đồ thị. Vui lòng thử lại sau.')
@app.post('/api/chat')
def chat(req:Request):
 bus=MessageBus();trace=bus.trace
 bus.register(Agent("Mô hình",lambda payload,b: models[payload["task"]].predict(payload["x"])[0]))
 bus.register(Agent("Truy xuất",lambda payload,b: retriever.search(payload)))
 bus.register(Agent("Đối chiếu",lambda payload,b: {"scope":payload.get("scope","Kiến thức có
  ↪  nguồn"),"allow_diagnosis":False}))
 def grounded_reply(payload,b):
  selected=payload[:2]
  if any(d['id']=='meal_example' for d in selected):selected=[d for d in selected if d['id']!='plate']
  return '\n\n'.join(d['text']+' ['+d['id']+']' for d in selected)
 bus.register(Agent('Phản hồi',grounded_reply))
 def msg(sender,receiver,kind,content):trace.append(dict(sender=sender,receiver=receiver,type=kind,con
  ↪  tent=content))
 msg('Người dùng','Điều phối','REQUEST',req.question or req.task)
 if req.task not in ['knowledge','classification','regression']:raise HTTPException(422,'Bài toán
  ↪  không hợp lệ')
 bus.register(Agent('An toàn',lambda payload,b:
  ↪  safety.classify(payload['question'],payload['history'])))
 assessment=bus.request('Điều phối','An toàn',{'question':req.question,'history':[h.model_dump() for h
  ↪  in req.history]})
 # Triage precedes model input checks, topic routing, Neo4j and language generation.
 if safety.protected(assessment):
  ids=['low','urgent'] if assessment.reason=='possible_hypoglycemia' else ['sick_day','urgent'] if
   ↪  assessment.reason in {'possible_dka','high_glucose_with_vomiting','high_glucose'} else
   ↪  ['medicine'] if assessment.medication_risk and assessment.level=='ROUTINE' else ['urgent'] if
   ↪  assessment.level=='EMERGENCY' else []
  selected=[d for i in ids for d in docs if d['id']==i]
  answer=safety.validate(assessment,clean_answer(safety.response(assessment)))
  msg('An toàn','Người dùng','INFORM',{'level':assessment.level,'reason':assessment.reason,'generation
   ↪  _bypassed':True,'validation':'protected-response'})
  return {'answer':answer,'safety':assessment.public(),'result':None,'trace':trace,'sources':[{'id':d[
   ↪  'id'],'title':d['title'],'excerpt':d['text'],'source':kb['sources'].get(d['source'],{})} for d
   ↪  in selected],'graph_evidence':[],'feedback_supported':False,'response_id':None,'response_mode':'
   ↪  rules','generator':'Luật an toàn cục
   ↪  bộ','suggestions':[],'context_used':assessment.context_used,'topic':'low' if
   ↪  assessment.reason=='possible_hypoglycemia' else 'urgent' if assessment.level=='EMERGENCY' else
   ↪  'medicine' if assessment.medication_risk else 'safety','tone':'urgent' if
   ↪  assessment.level!='ROUTINE' else 'normal'}
 result=None
 if req.task!='knowledge':
  fields=schema[req.task];missing=[f for f in fields if f not in req.values or req.values[f] in
   ↪  ['',None]]
  if missing:
   msg('Kiểm tra','Phản hồi','REQUEST','Bổ sung: '+', '.join(missing))
   return {'answer':'Cần bổ sung các trường: '+',
    ↪  '.join(missing),'trace':trace,'sources':[],'result':None}
  try:
   values=[float(req.values[f]) for f in fields]
   if not all(math.isfinite(v) for v in values):raise ValueError()
   if any(abs(v)>10000 for v in values):raise ValueError()
```

```
 except (ValueError,TypeError):raise HTTPException(422,'Các giá trị phải là số hữu hạn trong miền
  ↪ nhập liệu.')
 model=models[req.task];x=pd.DataFrame([values],columns=fields) if req.task=='classification' else
  ↪ np.array([values])
 pred=bus.request('Điều phối','Mô hình',{'task':req.task,'x':x})
 if req.task=='classification':
  result={'label':str(pred),'scores':dict(zip(map(str,model.classes_),map(float,model.predict_proba(x
   ↪ )[0]))),'scope':'Nhãn dữ liệu tổng hợp; không phải chẩn đoán hay xác suất mắc
   ↪ bệnh.'}
 else:result={'value':float(pred),'scope':'Điểm tiến triển bệnh nghiên cứu; không phải glucose hoặc
  ↪ HbA1c.'}
 msg('Mô hình','Đối chiếu','INFORM',json.dumps(result,ensure_ascii=False))
q=req.question or ('phân loại nhãn tổng hợp classification High Moderate Low' if
 ↪ req.task=='classification' else 'điểm tiến triển diabetes')
plan=route(q,[h.model_dump() for h in req.history])
plan=plan_turn(q,[h.model_dump() for h in req.history],plan)
if result is not None:
 plan.update(topic=req.task,ids=['classification_scope'] if req.task=='classification' else
  ↪ ['model'],override=None,intro='',fixed_sources=True,skip_retrieval=False,suggestions=['HbA1c là
  ↪ gì?','Tôi cần kiểm tra đường huyết thế nào?'])
msg('Hội thoại','Điều phối','INFORM',{'topic':plan['topic'],'context_used':plan['context_used']})
selected=bus.request('Điều phối','Truy xuất',plan)
graph_evidence=[]
graph_state='not-requested'
graph_agent=GraphKnowledgeAgent(kb,lambda:graph_driver(),database)
bus.register(Agent('Đồ thị tri thức',graph_agent.receive))
try:
 graph_result=bus.request('Điều phối','Đồ thị tri thức',{'document_ids':[d['id'] for d in selected]})
 graph_evidence=graph_result['facts'];graph_state=graph_result['storage']
except Exception:
 graph_state='unavailable'
 msg('Điều phối','Truy xuất','INFORM','Đồ thị chưa sẵn sàng; tiếp tục bằng văn bản KB đã có nguồn.')
# Learned workflow policy; rules constrain output when no evidence exists.
policy=json.loads((ROOT/'models/workflow_policy.json').read_text(encoding='utf-8'))
state=int(result is not None)*2+int(bool(selected)); action=int(np.argmax(policy['q'][state]))
if not selected:action=0
msg('Chính sách RL','Phản
 ↪ hồi','INFORM',{'state':state,'action':policy['actions'][action],'rule':'Thiếu bằng chứng thì yêu
 ↪ cầu làm rõ'})
answer=bus.request('Điều phối','Phản hồi',selected) if action in (1,2) else 'Tôi chưa tìm thấy bằng
 ↪ chứng phù hợp trong kho tri thức. Bạn hãy nêu rõ chỉ số hoặc chủ đề cần tìm hiểu.'
if plan['override'] is not None:answer=plan['override']
elif plan['intro']:answer=plan['intro']+'\n\n'+answer
if result:
 verdict=bus.request('Điều phối','Đối chiếu',result)
 answer=explain(req.task,result)
msg('Đối chiếu','Phản hồi','INFORM','Giữ đúng phạm vi mô hình, dẫn nguồn tài liệu; không suy ra chẩn
 ↪ đoán từ nhãn tổng hợp.')
response_mode="kb-rules"
feedback_id=None;presentation=0
if local_llm.enabled() and selected and not plan.get('guarded') and result is None:
 feedback_id,presentation=feedback.choose(plan['topic'],req.preferred_format)
 plan['format']=feedback.FORMATS[presentation]
 msg('Chính sách phản hồi','Điều phối','INFORM',{'strategy':plan['format'],'selected_by':'user' if
  ↪ req.preferred_format!='auto' else 'local-Q'})
if local_llm.enabled() and result is None:
 bus.register(Agent('Kiểm chứng phản hồi',local_llm.verify))
 bus.register(Agent('Ngôn ngữ cục bộ',local_llm.respond))
 try:
```

```
   generated=bus.request('Điều phối','Ngôn ngữ cục bộ',{'question':q,'plan':plan,'documents':selected,
    ↪  'graph_evidence':graph_evidence,'fallback':answer,'history':[h.model_dump() for h in
    ↪  req.history]})
   answer=generated['answer'];response_mode=generated['mode']
   msg('Kiểm chứng phản hồi','Điều
    ↪  phối','INFORM',{'mode':generated['mode'],'reason':generated['reason']})
  except Exception as exc:
   msg('Ngôn ngữ cục bộ','Điều phối','FAILURE',type(exc).__name__+'; sử dụng phản hồi KB')
 msg('Phản hồi','Người dùng','INFORM','Hoàn thành')
 if response_mode.startswith('qwen'):
  cited=set(re.findall(r'\[([a-z0-9_]+)\]',answer))
  selected=[d for d in selected if d['id'] in cited]
  graph_evidence=[r for r in graph_evidence if r['document'] in cited]
 answer=clean_answer(answer)
 answer=safety.validate(assessment,answer)
 return {'answer':answer,'safety':assessment.public(),'result':result,'trace':trace,'sources':[{'id':d
  ↪  ['id'],'title':d['title'],'excerpt':d['text'],'retrieval_score':d.get('retrieval_score'),'source'
  ↪  :kb.get('sources',{}).get(d.get('source'),{})} for d in
  ↪  selected],'graph_evidence':graph_evidence,'graph_status':graph_state,'graph_context_supplied':boo
  ↪  l(graph_evidence) and
  ↪  response_mode.startswith('qwen'),'feedback_supported':bool(feedback_id),'response_id':feedback_id
  ↪  ,'response_mode':response_mode,'generator':('Qwen3 cục bộ + RAG và KG' if graph_evidence and
  ↪  response_mode.startswith('qwen') else 'Qwen3 cục bộ + RAG' if response_mode.startswith('qwen')
  ↪  else 'KB và luật có nguồn'),'suggestions':plan['suggestions'],'context_used':plan['context_used']
  ↪  ,'topic':plan['topic'],'tone':plan['tone']}


@app.get('/api/knowledge')
def knowledge(q:str=''):
 from app.conversation import norm
 found=[d for d in docs if not q or norm(q) in norm(d['title']+' '+' '.join(d['terms'])+' '+d['text'])]
 return {'documents':[dict(d,source_info=kb['sources'].get(d['source'],{})) for d in
  ↪  found],'total':len(docs)}

@app.get('/api/status')
def status():
 model_name='Qwen3 4B · RAG local' if os.environ.get('GLUCOGUIDE_LLAMA_URL') else 'Qwen3 1.7B LoRA' if
  ↪  local_llm.enabled() else None
 try:
  g=graph();return {'neo4j':True,'nodes':len(g['nodes']),'edges':len(g['edges']),'documents':len(docs)
   ↪  ,'neo4j_browser':browser_url(),'local_llm':local_llm.enabled(),'model_name':model_name,'local_on
   ↪  ly':os.environ.get('GLUCOGUIDE_LOCAL_ONLY')=='1'}
 except HTTPException:return {'neo4j':False,'nodes':0,'edges':0,'documents':len(docs),'neo4j_browser':
  ↪  browser_url(),'local_llm':local_llm.enabled(),'model_name':model_name,'local_only':os.environ.get
  ↪  ('GLUCOGUIDE_LOCAL_ONLY')=='1'}

class FeedbackRequest(BaseModel):
 response_id:str=Field(min_length=10,max_length=80)
 helpful:bool
@app.post('/api/feedback')
def rate(req:FeedbackRequest):
 if not local_llm.enabled():raise HTTPException(404,'Phản hồi chỉ có ở bản local.')
 result=feedback.record(req.response_id,req.helpful)
 if result is None:raise HTTPException(409,'Lượt này đã được đánh giá hoặc đã hết hạn.')
 return result
```

A.7. app/safety.py

```
"""Offline conservative triage, independent of retrieval/generation. Not a diagnosis.

Uses the project's existing low/urgent/sick-day/medicine KB. The high-glucose
cutoff is an operational routing threshold, not a diagnostic fasting threshold.
No clinical model, downloads, socket operations or external services are used.
"""
import re
from dataclasses import dataclass, asdict
from app.retrieval import normalize

PATTERNS = {
 'unconscious': r'bat tinh|hon me|goi (?:mai |hoai )?(?:cung )?khong (?:day|tinh|tra loi|dap)|lay
  ↪  (?:mai )?khong (?:day|tinh)|khong danh thuc duoc|khong con biet gi',
 'seizure': r'co giat|giat (?:toan )?nguoi|tay chan giat lien tuc',
 'breathing': r'kho tho|tho (?:gap|doc|ho hen|hon hen|nhanh va sau)|khong tho (?:noi|duoc)|hut hoi du
  ↪  doi',
 'confused': r'lu lan|lo mo|noi (?:sang|mang)|khong nhan ra nguoi (?:than|nha)',
 'swallow': r'khong nuot duoc|nuot khong (?:noi|duoc)',
 'vomiting': r'non(?: oi| mua)?|oi mua|oi lien tuc|thuc an (?:va nuoc )?(?:cu )?trao ra|khong giu duoc
  ↪  (?:nuoc|thuc an)',
 'abdomen': r'dau bung|bung (?:dau|quan|quat)|dau (?:o |vung )?bung',
 'tremor': r'run tay|tay run|run (?:ban |lap cap)|chan tay run',
 'sweating': r'va mo hoi|va mo hoi|toat mo hoi|mo hoi (?:lanh|dam dia)|u[o]?t dam mo hoi',
 'dizzy': r'chong mat|choang vang|hoa mat|xay xam|ch[o]?ang v[a]?ng',
 'blood_urine': r'(?:tieu|dai|nuoc tieu)[^.!?;]{0,25}(?:ra mau|co mau|lan mau)|tieu mau|nuoc tieu
  ↪  (?:mau )?do',
}


def normalized(text):
 q=normalize(text)
 for bad,good in [('duong huyt','duong huyet'),('mo hoii','mo hoi'),('va mo hoi','va mo hoi'),('non
  ↪  oi','non oi'),('insullin','insulin')]:q=q.replace(bad,good)
 return q


def present(q, pattern):
 for m in re.finditer(r'(?<!\w)(?:'+pattern+r')(?!\w)',q):
  prefix=re.split(r'[,;.!?]|\b(?:nhung|tuy nhien)\b',q[:m.start()])[-1][-35:]
  if not re.search(r'\b(?:khong|chua|chang|het|khong
   ↪  con)\s+(?:bi\s+|co\s+|con\s+)?(?:\w+\s+){0,2}$',prefix):return True
 return False


def glucose(q):
 readings=[]
 # Units anchor the reading, so insulin dose/age/HbA1c cannot become glucose.
 for m in re.finditer(r'(\d+(?:[.,]\d+)?)\s*(mg\s*/?\s*dl|mmol\s*/?\s*l)\b',q):
  prefix=q[max(0,m.start()-40):m.start()]
  markers=list(re.finditer(r'ldl|hdl|cholesterol|triglyceride|duong huyet|glucose',prefix))
  if markers and markers[-1][0] not in {'duong huyet','glucose'}:continue
  if re.search(r'khong (?:phai|con)|hom qua|tuan truoc|truoc
   ↪  day',q[max(0,m.start()-35):m.start()]):continue
  value=float(m[1].replace(',','.'))*(18 if 'mmol' in m[2] else 1)
  if 0<value:readings.append(value)
 return readings
```

```
@dataclass
class Assessment:
 level:str='ROUTINE'
 medication_risk:bool=False
 reason:str='no_priority_rule'
 symptoms:tuple=()
 glucose_mg_dl:tuple=()
 context_used:bool=False
 def public(self):return asdict(self)


def classify(question, history=()):
 q=normalized(question)
 educational=bool(re.search(r'^(?:neu\b|gia su\b|vi du\b|dau hieu\b|trieu chung\b)',q.strip()))
 if re.search(r'\bla gi\s*[?.!]*$',q) and not re.search(r'toi (?:dang|bi)|me toi|hien tai|bay gio',q)
  ↪  and not (glucose(q) and any(present(q,p) for p in PATTERNS.values())):educational=True
 historical=bool(re.search(r'hom qua|tuan truoc|truoc day',q)) and bool(re.search(r'gio (?:da
  ↪  )?(?:het|on|binh thuong)|hien tai (?:da )?(?:het|on|binh thuong)',q))
 if educational or historical:return Assessment(reason='educational_or_resolved_history')
 symptoms=tuple(k for k,v in PATTERNS.items() if present(q,v))
 readings=glucose(q)
 prior=[normalized(h['content']) for h in history if h['role']=='user']
 relevant=prior[-1] if prior else ''
 used=False
 if not readings and len(q)<180 and re.search(r'gio|bay gio|van|toi dang|them',q) and
  ↪  set(symptoms)&{'vomiting','abdomen','breathing','tremor','sweating','dizzy'}:
  new_subject=re.search(r'\b(?:me toi|bo toi|con toi|vo toi|chong toi)\b',q)
  if not new_subject or new_subject[0] in relevant:
   readings=glucose(relevant);used=bool(readings)
 diabetes=bool(re.search(r'tieu duong|dai thao duong|insulin|duong huyet',q+' '+(relevant if used else
  ↪  '')))
 medication=bool(re.search(r'insulin|thuoc|metformin',q)) and present(q,r'tang(?: len| lieu)?|giam
  ↪  lieu|ngung(?: thuoc)?|dung thuoc|bo thuoc|bo lieu|tiem them|gap doi|doi lieu|tinh lieu|may don
  ↪  vi|bao nhieu don vi')
 data=dict(medication_risk=medication,symptoms=symptoms,glucose_mg_dl=tuple(readings),context_used=use
  ↪  d)
 if set(symptoms)&{'unconscious','seizure','breathing','confused','swallow'}:
  return Assessment('EMERGENCY',reason='severe_function_impairment',**data)
 high=any(v>=250 for v in readings)
 gi={'vomiting','abdomen'} <= set(symptoms)
 if gi and (high or diabetes):return Assessment('EMERGENCY',reason='possible_dka',**data)
 if high and 'vomiting' in symptoms:return
  ↪  Assessment('EMERGENCY',reason='high_glucose_with_vomiting',**data)
 low=any(v<70 for v in readings)
 cluster=len(set(symptoms)&{'tremor','sweating','dizzy'})>=2
 if low or (cluster and diabetes):return Assessment('URGENT',reason='possible_hypoglycemia',**data)
 if high:return Assessment('URGENT',reason='high_glucose',**data)
 if 'blood_urine' in symptoms:return Assessment('URGENT',reason='blood_in_urine',**data)
 return Assessment('ROUTINE',reason='medication_change' if medication else 'no_priority_rule',**data)


def protected(a):return a.level!='ROUTINE' or a.medication_risk


def response(a):
 if a.level=='EMERGENCY':
  answer='Hãy đến khoa cấp cứu ngay hoặc nhờ người bên cạnh gọi cấp cứu địa phương. Không tự lái xe và
   ↪  không trì hoãn để chờ chat hay bổ sung thời điểm đo.'
```

```
  if a.reason in {'possible_dka','high_glucose_with_vomiting'}:
   answer+=' Đường huyết cao kèm nôn hoặc đau bụng có thể là dấu hiệu nhiễm toan ceton do đái tháo
    ↪ đường (DKA) hoặc một tình trạng cấp tính khác; cần được đánh giá trực tiếp, chưa thể chẩn đoán
    ↪ chắc chắn qua chat.'
  else:answer+=' Khi người bệnh bất tỉnh, co giật, lơ mơ hoặc không nuốt được, không cho ăn uống qua
   ↪ miệng. Nhờ người ở cạnh hỗ trợ và làm theo hướng dẫn của nhân viên cấp cứu.'
 elif a.reason=='possible_hypoglycemia':
  answer='Các dấu hiệu này có thể là hạ đường huyết. Kiểm tra đường huyết ngay nếu có máy; không chờ
   ↪ chat nếu tình trạng nặng lên. Với người lớn còn tỉnh táo và nuốt được, nếu số đo dưới 70 mg/dL
   ↪ hoặc nghi hạ đường huyết mà chưa đo được, dùng 15 g carbohydrate hấp thu nhanh, đo lại sau 15
   ↪ phút và lặp lại nếu vẫn thấp. Nếu không cải thiện hoặc triệu chứng nặng hơn, cần trợ giúp y tế
   ↪ ngay. Nếu bất tỉnh, co giật, lơ mơ hoặc không nuốt được: nhờ người bên cạnh gọi cấp cứu, không
   ↪ cho ăn uống qua miệng. Trẻ em và thai kỳ cần kế hoạch xử trí riêng với nhóm chăm sóc.'
 elif a.reason=='high_glucose':
  answer='Số đường huyết bạn nêu đang cao; hãy liên hệ nhóm điều trị hoặc cơ sở y tế để được đánh giá
   ↪ sớm, nhất là nếu đo lại vẫn cao. Nếu có máy, kiểm tra lại và ghi kết quả, đơn vị, thời điểm cùng
   ↪ triệu chứng; theo kế hoạch xử trí ngày ốm đã được bác sĩ hướng dẫn nếu bạn có. Nếu nôn, đau
   ↪ bụng, khó thở, lơ mơ hoặc không giữ được nước uống, hãy đến khoa cấp cứu ngay. Không dùng ngưỡng
   ↪ xét nghiệm lúc đói để kết luận trường hợp này.'
 elif a.reason=='blood_in_urine':
  answer='Bạn nên được cơ sở y tế đánh giá sớm khi tiểu ra máu; không tự quy triệu chứng này cho tiểu
   ↪ đường. Chatbot chưa đủ căn cứ xác định nguyên nhân. Nếu kèm tình trạng lơ mơ, khó thở hoặc diễn
   ↪ biến nặng, cần trợ giúp cấp cứu ngay.'
 else:answer='Việc đổi hoặc tự ngừng thuốc cần trao đổi với bác sĩ hoặc dược sĩ đang theo dõi bạn; ghi
  ↪ lại tên thuốc, liều được kê, giờ dùng, kết quả đường huyết và triệu chứng để được hướng dẫn.'
 if a.medication_risk:
  answer+=' Không tự tăng, giảm, ngừng hoặc tiêm bù insulin/thuốc theo gợi ý trên chat. Đặc biệt, tự
   ↪ tăng insulin có thể gây hạ đường huyết nguy hiểm; mình không tính hay chỉ định liều cho bạn. Hãy
   ↪ liên hệ nhóm điều trị để xử lý số đo hiện tại. Nếu xuất hiện nôn, đau bụng, khó thở, lơ mơ hoặc
   ↪ không nuốt được, cần cấp cứu ngay.'
 return answer


def validate(a, answer):
 """Safety envelope is immutable after generation/cleanup, even on retrieval outage."""
 return response(a) if protected(a) else answer
```

A.8. app/retrieval.py

```
"""Local hybrid lexical retrieval: word + character TF-IDF and question aliases."""
import json
import re
import unicodedata
from pathlib import Path

import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

ROOT = Path(__file__).resolve().parents[1]
RETRIEVAL_CONFIG = {'word_ngram_range':[1,2], 'char_ngram_range':[3,5],
                    'word_weight':.55, 'char_weight':.45, 'minimum_score':.15,
                    'relative_cutoff':.58, 'max_documents':4,
                    'phrase_boost':.025, 'max_phrase_matches':3, 'routing_boost':.025}


def normalize(text):
    return ''.join(c for c in unicodedata.normalize('NFD', text.lower().replace('đ', 'd'))
```

```
                  if unicodedata.category(c) != 'Mn')


def load_knowledge():
    kb = json.loads((ROOT / 'data/knowledge.json').read_text(encoding='utf-8'))
    extra = json.loads((ROOT / 'data/knowledge_dialogue.json').read_text(encoding='utf-8'))
    kb['documents'] += extra['documents']
    kb['sources'].update(extra['sources'])
    assert len({d['id'] for d in kb['documents']}) == len(kb['documents'])
    return kb


class Retriever:
    def __init__(self, documents):
        self.config = dict(RETRIEVAL_CONFIG)
        self.docs = documents
        self.by_id = {d['id']: d for d in documents}
        texts = [normalize(' '.join([d['title']] * 2 + d['terms'] + d.get('questions', []) +
        ↪  [d['text']])) for d in documents]
        self.word = TfidfVectorizer(ngram_range=tuple(self.config['word_ngram_range']),
        ↪  sublinear_tf=True)
        self.char = TfidfVectorizer(analyzer='char_wb',
        ↪  ngram_range=tuple(self.config['char_ngram_range']), sublinear_tf=True)
        self.words = self.word.fit_transform(texts)
        self.chars = self.char.fit_transform(texts)

    def search(self, plan):
        if plan.get('skip_retrieval'):
            return []
        if plan.get('fixed_sources'):
            return [dict(self.by_id[i], retrieval_score=1.0) for i in plan['ids'] if i in self.by_id]
        query = normalize(plan['query'])
        scores = (self.config['word_weight'] * (self.words @
        ↪  self.word.transform([query]).T).toarray().ravel()
              + self.config['char_weight'] * (self.chars @
               ↪  self.char.transform([query]).T).toarray().ravel())
        for i, doc in enumerate(self.docs):
            # Domain terms boost exact whole phrases, not substrings such as "com" in computer.
            matches = sum(bool(re.search(r'(?<!\w)' + re.escape(normalize(term)) + r'(?!\w)', query))
                          for term in doc['terms'] if len(normalize(term)) >= 4)
            scores[i] += min(matches, self.config['max_phrase_matches']) * self.config['phrase_boost']
            if doc['id'] in plan.get('ids', []):
                scores[i] += self.config['routing_boost']
            if doc.get('topics') and plan['topic'] in doc['topics']:
                scores[i] += self.config['routing_boost']
            if doc['source'] in {'dataset', 'classification_data'} and plan['topic'] not in {'model',
            ↪  'classification', 'meanpressure', 'demographics'}:
                scores[i] = 0  # Never mix research features into ordinary patient questions.
        maximum = float(scores.max())
        if maximum < self.config['minimum_score']:
            return []
        order = np.argsort(-scores)
        selected = [dict(self.docs[i], retrieval_score=round(float(scores[i]), 4))
                    for i in order[:self.config['max_documents']] if scores[i] >=
                     ↪  max(self.config['minimum_score'], maximum * self.config['relative_cutoff'])]
        return selected
```

A.9. app/graph_knowledge.py

```
"""Read-only, source-checked graph evidence for the existing local agent workflow."""
from collections import defaultdict

FACT_QUERY = '''MATCH (a:Concept)-[r:RELATED_TO]->(b:Concept)
WHERE r.document IN $ids
RETURN a.name AS source,r.label AS relation,b.name AS target,
       r.document AS document,r.evidence AS evidence,r.source_url AS source_url
ORDER BY document,source,relation,target LIMIT 48'''


def ground_facts(rows, documents, sources=None, limit=12):
    """Allow only exact KB triples with their original excerpt and source URL."""
    by_id = {d['id']: d for d in documents}
    accepted, rejected, seen = defaultdict(list), 0, set()
    for row in rows:
        row = dict(row)
        doc = by_id.get(row.get('document'))
        triple = [row.get('source'), row.get('relation'), row.get('target')]
        valid = all(k in row for k in
        ↪  ('source','relation','target','document','evidence','source_url')) and doc is not None and
        ↪  triple in doc.get('triples', []) and row.get('evidence') == doc['text']
        if valid and sources is not None:
            valid = row.get('source_url') == sources[doc['source']]['url']
        if not valid:
            rejected += 1
            continue
        key = (doc['id'], *triple)
        if key in seen:
            continue
        seen.add(key)
        accepted[doc['id']].append({k: row[k] for k in ('source', 'relation', 'target', 'document',
        ↪  'evidence', 'source_url')})
    # Round-robin prevents the first document from occupying the whole context.
    facts = []
    while len(facts) < limit and any(accepted.values()):
        for doc in documents:
            if accepted[doc['id']] and len(facts) < limit:
                facts.append(accepted[doc['id']].pop(0))
    return facts, rejected


class GraphKnowledgeAgent:
    def __init__(self, knowledge, driver_factory, database, storage='Neo4j'):
        self.knowledge = knowledge
        self.by_id = {d['id']: d for d in knowledge['documents']}
        self.driver_factory, self.database = driver_factory, database
        self.storage = storage

    def receive(self, payload, bus):
        ids = list(dict.fromkeys(i for i in payload['document_ids'] if i in self.by_id))[:4]
        if not ids:
            return {'facts': [], 'storage': 'not-requested', 'rejected': 0}
        with self.driver_factory() as driver:
            rows = driver.execute_query(FACT_QUERY, ids=ids, database_=self.database()).records
        facts, rejected = ground_facts(rows, [self.by_id[i] for i in ids], self.knowledge['sources'])
        return {'facts': facts, 'storage': self.storage, 'rejected': rejected, 'document_ids': ids}
```

```
def prompt_facts(payload):
    facts, _ = ground_facts(payload.get('graph_evidence', []), payload['documents'])
    return '\n'.join('['+r['document']+'] '+r['source']+' → '+r['relation']+' → '+r['target'] for r in
    ↪  facts)
```

A.10. scripts/query_knowledge_export.py

```
"""Query a trusted exported KB/KG/RAG bundle using only local files."""
import argparse
import hashlib
import json
from pathlib import Path
import sys
from types import SimpleNamespace

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
from app.agents import Agent, MessageBus
from app.conversation import route
from app.dialogue import plan_turn
from app.graph_knowledge import GraphKnowledgeAgent
from app import safety


def verify_bundle(folder):
    folder = Path(folder).resolve()
    manifest = json.loads((folder/'manifest.json').read_text(encoding='utf-8'))
    for relative, entry in manifest['files'].items():
        path = (folder/relative).resolve()
        if not path.is_relative_to(folder) or not path.is_file():
            raise ValueError('Invalid bundle path: '+relative)
        if path.stat().st_size != entry['bytes'] or hashlib.sha256(path.read_bytes()).hexdigest() !=
        ↪  entry['sha256']:
            raise ValueError('Checksum mismatch: '+relative)
    return manifest


class SnapshotDriver:
    def __init__(self, rows): self.rows = rows
    def __enter__(self): return self
    def __exit__(self, *args): return False
    def execute_query(self, query, *, ids, database_):
        return SimpleNamespace(records=[r for r in self.rows if r['document'] in ids])


def query_bundle(folder, question, history=None):
    import joblib
    folder = Path(folder).resolve()
    verify_bundle(folder)
    kb = json.loads((folder/'kb/knowledge.json').read_text(encoding='utf-8'))
    assessment = safety.classify(question, history or [])
    if safety.protected(assessment):
        return {'answer':safety.response(assessment), 'safety':assessment.public(),
        ↪  'mode':'safety-rules', 'sources':[], 'graph_evidence':[], 'trace':[]}
    # Load only a self-produced, checksum-checked artifact, never an Internet pickle.
    retriever = joblib.load(folder/'rag/index.joblib')
    graph = json.loads((folder/'kg/graph.json').read_text(encoding='utf-8'))
    plan = plan_turn(question, history or [], route(question, history or []))
```

```
    bus = MessageBus()
    bus.register(Agent('Truy xuất', lambda p,b: retriever.search(p)))
    kg = GraphKnowledgeAgent(kb, lambda: SnapshotDriver(graph['relationships']), lambda:'snapshot',
     ↪  storage='exported-snapshot')
    bus.register(Agent('Đồ thị tri thức', kg.receive))
    bus.register(Agent('Tổng hợp căn cứ', lambda p,b: '\n\n'.join(d['text'] for d in p)))
    selected = bus.request('Điều phối','Truy xuất',plan)
    facts = bus.request('Điều phối','Đồ thị tri thức',{'document_ids':[d['id'] for d in selected]})
    answer = plan.get('override') or (bus.request('Điều phối','Tổng hợp căn cứ',selected[:2]) if
     ↪  selected else 'Chưa có tài liệu phù hợp trong kho.')
    return {'answer':answer, 'mode':'snapshot-evidence-query', 'topic':plan['topic'],
            'sources':[{'id':d['id'],'score':d.get('retrieval_score')} for d in selected],
            'graph_evidence':facts['facts'], 'storage':facts['storage'], 'trace':bus.trace}


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument('folder', type=Path)
    parser.add_argument('--question', required=True)
    parser.add_argument('--output', type=Path)
    args = parser.parse_args()
    result = json.dumps(query_bundle(args.folder,args.question),ensure_ascii=False,indent=2)
    if args.output: args.output.write_text(result,encoding='utf-8')
    else:
        sys.stdout.reconfigure(encoding='utf-8')
        print(result)
```

A.11. scripts/test_scenarios.py

```
from pathlib import Path
import json
from fastapi.testclient import TestClient
from unittest.mock import patch
from app.main import app
r=Path(__file__).resolve().parents[1];c=TestClient(app);examples=json.loads((r/'data/examples.json').r
 ↪  ead_text())
cases=[('knowledge',{'question':'HbA1c là
 ↪  gì?'},200),('unknown',{'question':'zzzxxyy'},200),('classification',{'task':'classification','valu
 ↪  es':examples['classification']},200),('regression',{'task':'regression','values':examples['regress
 ↪  ion']},200),('missing',{'task':'classification','values':{}},200),('invalid',{'task':'classificati
 ↪  on','values':{**examples['classification'],'Age':'nan'}},422),('bad_task',{'task':'unknown'},422)]
results=[]
for name,payload,status in cases:
 response=c.post('/api/chat',json=payload);body=response.json();assert
  ↪  response.status_code==status,(name,body)
 if name=='unknown':assert body['sources']==[]
 if name=='knowledge':assert body['sources'][0]['id']=='a1c' and body['graph_evidence']
 if name=='classification':assert body['result']['label'] in ['High','Moderate','Low']
 if name=='regression':assert isinstance(body['result']['value'],float)
 if name=='missing':assert body['result'] is None
 results.append({'case':name,'status':status,'passed':True,'response':body})
with patch('app.main.graph_driver',side_effect=RuntimeError('controlled outage')):
 response=c.post('/api/chat',json={'question':'HbA1c'});body=response.json();assert
  ↪  response.status_code==200 and any(t['type']=='FAILURE' for t in body['trace'])
 assert c.get('/api/graph').status_code==503
 results.append({'case':'controlled_neo4j_outage','passed':True,'response':body})
```

```
g=c.get('/api/graph').json();assert g['storage']=='Neo4j' and len(g['nodes'])>0
(r/'results/api_scenarios.json').write_text(json.dumps(results,ensure_ascii=False,indent=2),encoding='
↪ utf-8')
print(json.dumps({'passed':len(results),'nodes':len(g['nodes']),'edges':len(g['edges'])}))
```

A.12. scripts/test_chatbot.py

```
from pathlib import Path
import json
from fastapi.testclient import TestClient
from app.main import app
c=TestClient(app);results=[]
cases=[
 ('blood_pressure_single',{'question':'huyết áp 150 có ổn không'},lambda b:b['topic']=='bloodpressure'
 ↪ and 'số trên' in b['answer'] and all(x['id']=='bloodpressure' for x in b['sources']) and 'bp của
 ↪ bộ' not in b['answer']),
 ('blood_pressure_pair',{'question':'huyết áp 150/90 có cao không'},lambda b:'150/90' in b['answer']
 ↪ and 'cao theo' in b['answer']),
 ('blood_pressure_followup',{'question':'số dưới là 90','history':[{'role':'user','content':'huyết áp
 ↪ 150 có ổn không'}]},lambda b:b['context_used'] and '150/90' in b['answer']),
 ('blood_pressure_severe',{'question':'huyết áp 190/125'},lambda b:b['tone']=='urgent' and 'cấp cứu'
 ↪ in b['answer']),
 ('blood_pressure_invalid',{'question':'huyết áp 90/150'},lambda b:'kiểm tra lại' in b['answer']),
 ('blood_pressure_home',{'question':'đo huyết áp tại nhà thế nào'},lambda b:'nghỉ yên 5 phút' in
 ↪ b['answer']),
 ('mean_pressure_explicit',{'question':'trường bp là huyết áp trung bình à'},lambda
 ↪ b:b['sources'][0]['id']=='meanpressure'),
 ('substring_food',{'question':'computer là gì'},lambda b:b['topic']!='meal'),
 ('unit_followup',{'question':'mmol/L, lúc đói','history':[{'role':'user','content':'đường huyết
 ↪ 7'}]},lambda b:b['context_used'] and '126' in b['answer']),
 ('detail_followup',{'question':'giải thích chi tiết thêm','history':[{'role':'user','content':'HbA1c
 ↪ 6,2%'}]},lambda b:b['context_used'] and 'Bước tiếp theo' in b['answer']),
 ('multiple_measurements',{'question':'HbA1c 6,2%, đường huyết lúc đói 7 mmol/L'},lambda b:'6.2%' in
 ↪ b['answer'] and '126' in b['answer'] and set(s['id'] for s in b['sources'])=={'a1c','testing'}),
 ('multiple_low',{'question':'HbA1c 6,2%, đường huyết 55 mg/dL'},lambda b:b['tone']=='urgent'),
 ('long_history',{'question':'Vậy 7,1% thì sao?','history':[{'role':'user','content':'HbA1c là
 ↪ gì?'}]*20},lambda b:b['context_used'] and '7.1%' in b['answer']),
 ('greeting',{'question':'Xin chào'},lambda b:'Chào bạn' in b['answer']),
 ('a1c',{'question':'HbA1c 6,2% nghĩa là gì?'},lambda b:'tiền đái tháo đường' in b['answer'] and
 ↪ b['sources'][0]['id']=='a1c'),
 ('followup',{'question':'Vậy 7,1% thì sao?','history':[{'role':'user','content':'HbA1c là
 ↪ gì?'}]},lambda b:b['context_used'] and '7.1%' in b['answer']),
 ('continued_context',{'question':'Vậy 8% thì sao?','history':[{'role':'user','content':'HbA1c là
 ↪ gì?'},{'role':'assistant','content':'Tham khảo NIDDK'},{'role':'user','content':'Vậy 7,1% thì
 ↪ sao?'}]},lambda b:b['context_used'] and '8%' in b['answer']),
 ('no_unit',{'question':'HbA1c 7 nghĩa là gì?'},lambda b:'xác nhận' in b['answer']),
 ('glucose_unit',{'question':'Đường huyết 7 thì sao?'},lambda b:'bổ sung đơn vị' in b['answer']),
 ('fasting',{'question':'Đường huyết lúc đói 7 mmol/L'},lambda b:'126' in b['answer'] and '8 giờ' in
 ↪ b['answer']),
 ('after_meal',{'question':'Đường huyết sau ăn 7 mmol/L'},lambda b:'chưa áp ngưỡng lúc đói' in
 ↪ b['answer']),
 ('hypoglycemia',{'question':'Đường huyết hiện tại 55 mg/dL'},lambda b:b['tone']=='urgent' and
 ↪ b['sources'][0]['id']=='low'),
 ('emergency',{'question':'Bố tôi đang bất tỉnh'},lambda b:b['tone']=='urgent' and
 ↪ b['sources'][0]['id']=='urgent'),
 ('education',{'question':'Dấu hiệu hạ đường huyết là gì?'},lambda b:b['topic']=='low' and
 ↪ b['tone']=='normal'),
```

```
 ('dose',{'question':'Tôi nên tiêm bao nhiêu đơn vị insulin?'},lambda b:b['topic']=='medicine' and
  ↪  b['safety']['medication_risk'] and 'không tính hay chỉ định liều' in b['answer']),
 ('pregnancy',{'question':'Tôi mang thai HbA1c 6,2%'},lambda b:'không áp bảng ngưỡng' in b['answer']
  ↪  and '6.2%' not in b['answer']),
 ('food',{'question':'Có cần bỏ cơm hoàn toàn không?'},lambda b:any(s['id']=='carb' for s in
  ↪  b['sources'])),
 ('self_identity',{'question':'bạn là ai','history':[{'role':'user','content':'Đường huyết là
  ↪  gì?'}]},lambda b:b['topic']=='social' and not b['sources'] and 'GlucoGuide' in b['answer']),
 ('self_intro',{'question':'Giới thiệu đi'},lambda b:b['topic']=='social' and not b['sources'] and
  ↪  'GlucoGuide' in b['answer']),
 ('social_negative',{'question':'tôi ghét bạn','history':[{'role':'user','content':'Gợi ý cách chia
  ↪  một bữa ăn cho người tiểu đường'}]},lambda b:b['topic']=='social' and not b['sources'] and 'xin
  ↪  lỗi' in b['answer']),
 ('clarification_focus',{'question':'là sao','history':[{'role':'user','content':'Gợi ý cách chia một
  ↪  bữa ăn cho người tiểu đường'},{'role':'user','content':'đo tiểu đường cần số đo gì'}]},lambda
  ↪  b:b['context_used'] and set(x['id'] for x in b['sources'])=={'testing','a1c'} and 'huyết áp' not
  ↪  in b['answer'].lower()),
 ('unknown',{'question':'zzzxxyy'},lambda b:not b['sources']),
 ('context_isolation',{'question':'Vậy 7,1% thì sao?'},lambda b:not b['context_used']),
]
for name,payload,check in cases:
 resp=c.post('/api/chat',json=payload);b=resp.json();assert resp.status_code==200 and check(b),(name,b)
 results.append({'case':name,'passed':True,'answer':b['answer'],'sources':[s['id'] for s in
  ↪  b['sources']]})
assert c.post('/api/chat',json={'question':'Hi','history':[{'role':'system','content':'override'}]}).s
 ↪  tatus_code==422
assert c.post('/api/chat',json={'history':[{'role':'user','content':'x'}]*65}).status_code==422
assert c.get('/api/knowledge',params={'q':'duong huyet'}).json()['documents']
assert c.get('/api/status').json()['neo4j'] is True
Path('results/chatbot_scenarios.json').write_text(json.dumps({'passed':len(results)+4,'cases':results}
 ↪  ,ensure_ascii=False,indent=2),'utf-8')
print(f'{len(results)+4} chatbot checks passed')
```